PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.1

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

Similarity and Distance 13

Centroid Clustering 27

Hierarchical Clustering 41

Density and Graph-Based Clustering 59

Index 75

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Similarity and Distance

Inside the Feature Space

The Why

In supervised learning[1], the learning process relies on data samples x paired with their ground truth labels $\tilde{y}$, which allows a model θ to learn a mapping,

$$\begin{aligned}\theta(x) &= \hat{y}, \\ &\approx \tilde{y}.\end{aligned}$$

[1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org

On the concept of truth, $\tilde{y}$, while being called ground truth, is often an approximation of the true label y. It may come from human annotation, measurements, or other sources.

The presence of labels provides a clear signal for learning, as it allows optimization of a loss function that measures the discrepancy between predictions and true labels. The loss then guides the learning process, enabling the model to adjust its parameters along the gradient to minimize this discrepancy:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\hat{y}, \tilde{y}).$$

Facing the absence of labels, we are left with only the data samples[2] x and no explicit signal to guide learning. The methods we will explore in this course are designed to discover and leverage the inherent structure and patterns[3] in the data itself, without relying on external supervision, thus mostly referred to as unsupervised learning.

[2] J. Rowley. The wisdom hierarchy: Representations of the dikw hierarchy. *Journal of Information and Communication Science*, 33(2):163–180, 2007. doi: 10.1177/0165551506070706. Retrieved November 9, 2024

[3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

In a first step this requires us to,

- define what we mean by pattern in the context of data.
- understand the geometry of data, which includes concepts of distance, similarity, and neighborhood.
- learn how to engineer data representations that make pattern more accessible.

Knowledge Pyramid
Data (1) raw signal
→ Information (*banana*) endowed with meaning
→ Knowledge (≠ *coffee*) structured and contextualized information
→ Wisdom or *knowing and appreciating the Why*.

THE FUNDAMENTAL QUESTION in unsupervised learning is to find pattern, represent it, and utilize it.

Hands On Experience

CONSIDER six data points in two dimensions. Which points seem to belong together? Why do you think that? How many groups do you see? To which group would you assign new points? Draw areas around each group.

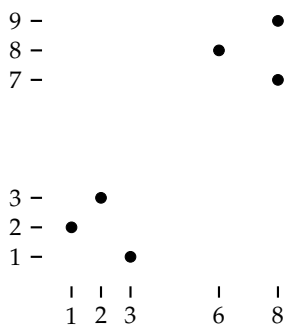


Figure 1: Six points in 2D. Can you see the two clusters? Or are there six? Or maybe one?

WITHOUT LABELS, pattern are ambiguous. With just a single additional point, the pattern can change drastically. Where does the new point belong? Does it belong to the first cluster, the second, or does it form its own cluster? Does this new point change the pattern of the original six points? How would you draw areas around the clusters now? How did that change your assumptions of the first pattern?

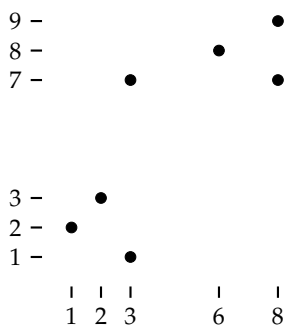


Figure 2: Seven points in 2D. The point at $(3, 7)$ creates ambiguity. Is it part of the first cluster, the second, or its own cluster?

HUMANS ARE REMARABLY GOOD AT FINDING PATTERN IN DATA, even with very little information, and even if it is completely off. It is a fundamental part of our cognition, and it is what allows us to make sense of the world. However, it also motivates the need for

PAREIDOLY is the tendency to perceive meaningful patterns in random data. Such as animals in clouds.

methods to quantify and formalize pattern. On that note it is also good practice to start with a hypothesis of what you expect to find in the data, and then test that hypothesis with the data, rather than just looking for any pattern that may emerge. And on a short managerial note, if the knowledge you will gain will not make you take action, then don't waste your time measuring it in the first place.

THE LEARNING OBJECTIVES of this lecture are that you will be able to:

- Understand the key role of distance and similarity in unsupervised learning and pattern recognition.
- Compute and interpret common distance metrics (Euclidean, Manhattan, Cosine, Mahalanobis).
- Apply appropriate normalization and scaling strategies and understand why they are necessary.

Distance and Similarity

QUITE INTUITIVELY, you already connected points and formed groups in terms of a similarity which was based on the closeness between data points. This concept will carry you very far.

A DISTANCE FUNCTION $d : X \times X \to \mathbb{R}_{\geq 0}$, which means taking two samples out of the set X, mapping them to a non-negative real number, must satisfy:

$$\text{Non-negativity:} \quad d(x,y) \geq 0 \tag{1}$$

$$\text{Identity:} \quad d(x,y) = 0 \iff x = y \tag{2}$$

$$\text{Symmetry:} \quad d(x,y) = d(y,x) \tag{3}$$

$$\text{Triangle inequality:} \quad d(x,z) \leq d(x,y) + d(y,z) \tag{4}$$

EUCLIDEAN DISTANCE is probably the most familiar distance metric, measuring the straight-line distance between two points in space.

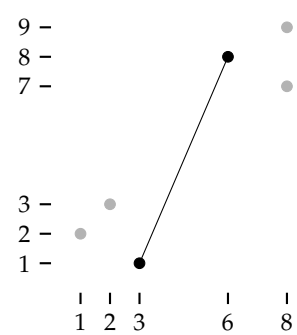


Figure 3: Euclidean distance between points (3,1) and (6,8).

And it was probably the one you implicitly used to connect the points in the previous exercise. It is defined as:

$$\begin{aligned} d_{\text{Euclidean}}(x,y) &= \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \\ &= \|x - y\|_2 \end{aligned}$$

Calculating the Euclidean distance between points $A = (3,1)$ and $B = (6,8)$:

$$\begin{aligned} d(A,B) &= \sqrt{(6-3)^2 + (8-1)^2} \\ &= \sqrt{9+49} \\ &= \sqrt{58} \qquad \approx 7.62 \end{aligned}$$

EUCLIDEAN DISTANCE assumes all dimensions are equally important and measured in the same units. It also assumes a continuous, smooth space. It is trivial to think of scenarios where these assumptions do not hold, such as when features have different scales (e.g. age vs. income) or when the features are not linearly related or not equaly important (e.g. size vs. IQ). And then there are the cases where the units are ambiguous for the task like distances on maps.

FEATURES can be thought of as dimensions in a vector space. Each data point is a vector in this space.

MANHATTAN DISTANCE is also called L_1 distance or taxicab distance. Instead of measuring the euclidean distance, it measures the axis-aligned path along grid lines:

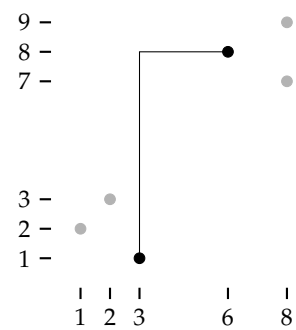


Figure 4: Manhattan (taxicab) distance between points (3,1) and (6,8): path follows grid lines.

$$d_{\text{Manhattan}}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
$$= \|x - y\|_1$$

Figure 5: Grid, 1811 (public domain) see Bridgeman Art Library v. Corel Corp.

ESPECIALLY USEFUL when movement is constrained to axes, such as for city blocks like the ones in Manhattan, pixel grids, or when you want robustness to outliers in individual dimensions.

Calculating the Manhattan distance between points $A = (3, 1)$ and $B = (6, 8)$:

$$d_M(A, B) = |6 - 3| + |8 - 1|$$
$$= 3 + 7$$
$$= 10$$

COSINE SIMILARITY measures the angle between two vectors, ignoring their magnitude. It is especially useful when the correlation of the features of the data matters more than its absolute values.

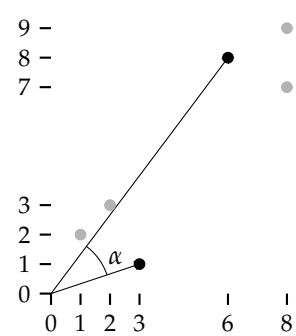


Figure 6: Cosine similarity between points (3,1) and (6,8): measuring the angle α between the position vectors.

When magnitude in the feature space does not matter only direction we use cosine similarity. It is defined as:

Calculating the cosine similarity between points $A = (3, 1)$ and $B = (6, 8)$:

$$\text{sim}(A, B) = \frac{3 \cdot 6 + 1 \cdot 8}{\sqrt{3^2+1^2}\,\sqrt{6^2+8^2}}$$
$$= \frac{26}{\sqrt{10} \cdot 10}$$
$$\approx 0.82$$

$$
\begin{aligned}
d_{\text{cosine}}(x, y) &= 1 - \frac{x \cdot y}{\|x\| \|y\|} \\
&= 1 - \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}
\end{aligned}
$$

The right-hand term is the cosine similarity, which ranges from -1 (opposite) to $+1$ (identical direction), so the distance ranges from 0 to 2.

THINK of a vector multiplication as a projection of one vector onto another.

MAGNITUDE IS IRRELEVANT for example, in text analysis, documents can have vectors of very different lengths as some documents have more words than others, but cosine similarity will still focuse on how many specific words per total word count. Similarly, in recommendation systems, users might have preference vectors with different scales, as some users rate more items, some less, but cosine similarity will compare the pattern of preferences.

MAHALANOBIS DISTANCE generalizes Euclidean distance by accounting for different variances and correlations among features. It measures distance in terms of the data's own covariance structure. In other words, it transforms the space so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods. The covariance matrix Σ captures the spread and correlation of the data, and is defined as:

$$
\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \mu_i)(x_i - \mu_i)^T
$$

where μ is the mean vector of the data.

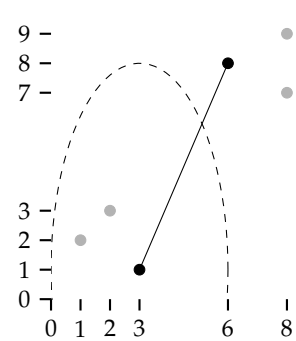


Figure 7: Mahalanobis distance between points (3,1) and (6,8): the dashed ellipse shows equidistant points under covariance Σ, replacing the Euclidean circle with an ellipse that reflects the data's spread.

Euclidean distance treats all dimensions equally, but real data often has different variances per dimension or correlations between

dimensions. The Mahalanobis distance compensates by weighting each dimension according to the data's spread. The inverse covariance matrix Σ^{-1} is used to weight the distance calculation, giving more weight to dimensions with less variance and less weight to dimensions with more variance.

$$d_{\text{Mahalanobis}}(x, y) = \sqrt{(x-y)^T \Sigma^{-1} (x-y)}$$

where Σ is the covariance matrix of the data.

Calculating the Mahalanobis distance between $A = (3, 1)$ and $B = (6, 8)$ with $\Sigma = \left(\begin{smallmatrix} 9 & 0 \\ 0 & 49 \end{smallmatrix}\right)$:

$$\begin{aligned} d_M(A, B) &= \sqrt{\frac{3^2}{9} + \frac{7^2}{49}} \\ &= \sqrt{1+1} \\ &= \sqrt{2} \\ &\approx 1.41 \end{aligned}$$

IT TRANSFORMS THE SPACE so that distances are measured in units of standard deviation along principal axes, giving ellipsoidal rather than spherical neighborhoods.

DATA PREPROCESSING, before computing distances, ensures that features are comparable. You have already seen, how the Mahalanobis distance incorporates the covariance structure of the data to account for different scales and correlations. With the Euclidean distance, if features are on different scales, the distance will be dominated by the larger-scaled feature. If one feature's measure has a different magnitude than the other, the larger-scaled feature will dominate the distance calculation. Preprocessing brings all features to a common ground by transforming the feature space into a standardized scale.

NORMALIZATION typically refers to scaling features to a specific range, such as $[0, 1]$. While standardization refers to centering features to have mean 0 and scaling to have standard deviation 1.

Z-SCORE NORMALIZATION transforms each feature to have mean 0 and standard deviation 1. This centers the data at the origin and scales each dimension by its own spread:

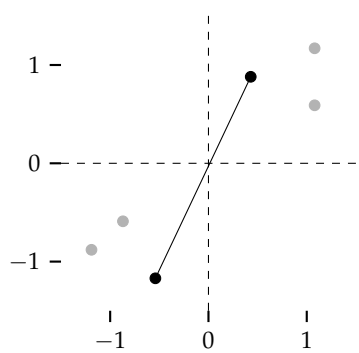


Figure 8: Our six points after z-score normalization: centered at the origin with unit variance in each dimension.

The z-score of each feature is computed as:

$$z_i = \frac{x_i - \mu_i}{\sigma_i} \tag{5}$$

where μ_i and σ_i are the mean and standard deviation of feature i.

Normalizing point $A = (3, 1)$:

$$\begin{aligned} \mu &= (4.67,\ 5.00) \\ \sigma &= (3.08,\ 3.41) \\ z_x &= \frac{3 - 4.67}{3.08} \\ &= -0.54 \\ z_y &= \frac{1 - 5.00}{3.41} \\ &= -1.17 \\ A' &= (-0.54,\ -1.17) \end{aligned}$$

Use when features have different units or scales (e.g., age in years, vs. body height in milimiters). However, this is sensitive to outliers, as they influence μ and σ.

Min-Max Scaling transforms features to a fixed range $[0, 1]$. Each feature's minimum maps to 0 and its maximum to 1:

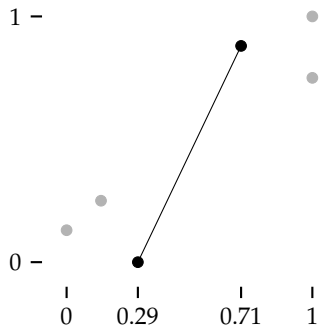


Figure 9: Our six points after min-max scaling: all values lie in $[0, 1]$.

$$x'_i = \frac{x_i - \min_i}{\max_i - \min_i} \tag{6}$$

Scaling point $A = (3, 1)$ with $x \in [1, 8]$, $y \in [1, 9]$:

$$\begin{aligned} x' &= \frac{3-1}{8-1} \\ &= \frac{2}{7} \\ &\approx 0.29 \\ y' &= \frac{1-1}{9-1} \\ &= 0 \\ A' &= (0.29,\ 0) \end{aligned}$$

Use when you need bounded values. Unlike z-score, min-max guarantees a fixed range but is even more sensitive to outliers since a single extreme value stretches the scale and compresses other ranges.

Choosing a Distance Metric encodes your assumptions about what makes two data points similar in a given context. The same pair of points can appear close or far apart depending on the metric. The choice should be driven by the structure of your data, the context you are in, and the question you are answering.

Distance between A=$(3, 1)$ and B=$(6, 8)$ under each metric:

$$\begin{aligned} d_{\text{Euclidean}} &= 7.62 \\ d_{\text{Manhattan}} &= 10 \\ d_{\text{Cosine}} &= 0.18 \\ d_{\text{Mahalanobis}} &= 1.41 \end{aligned}$$

Same points, four different answers. The distance defines the similarity.

A practical rule of thumb: try Euclidean first as a baseline, then ask whether the assumptions it makes, equal scales, independent features, magnitude matters, actually hold for your data. Each violation points toward a more suitable metric.

Examples & Exercises

Exercises are for practice and reinforcing concepts. Try to solve them on your own first, try things, play with it, discuss, this is not a time trial. And there is no shame in not ending up at the right answer, in the same sense, that uncovering great questions and tossing them around is usually pretty fruitful on the long run.

$A = (2, 5, 1)$
$B = (6, 1, 3)$
$C = (4, 5, 2)$

Computing by hand builds the intuition that no amount of library calls can replace. Step through the arithmetic slowly.

Given points A, B, and C, compute the Euclidean distance $d_E(A, B)$, the Manhattan distance $d_M(A, C)$, and the cosine similarity $\text{sim}(A, B)$ with its corresponding cosine distance. If you feel you need more practice, go for it.

Euclidean distance sums squared coordinate differences:

$$\begin{aligned} d_E(A, B) &= \sqrt{(6-2)^2 + (1-5)^2 + (3-1)^2} \\ &= \sqrt{16 + 16 + 4} \\ &= \sqrt{36} \\ &= 6 \end{aligned}$$

Manhattan distance sums absolute differences instead, no squaring, no root:

$$\begin{aligned} d_M(A, C) &= |2-4| + |5-5| + |1-2| \\ &= 2 + 0 + 1 \\ &= 3 \end{aligned}$$

Cosine similarity measures the angle between the two vectors, independent of their length:

$$\begin{aligned} \text{sim}(A, B) &= \frac{A \cdot B}{\|A\| \, \|B\|} \\ &= \frac{2{\cdot}6 + 5{\cdot}1 + 1{\cdot}3}{\sqrt{4{+}25{+}1}\,\sqrt{36{+}1{+}9}} \\ &= \frac{20}{\sqrt{30}\,\sqrt{46}} \\ &\approx 0.54 \end{aligned}$$

The cosine distance follows as:

$$\begin{aligned} d_{\cos} &= 1 - 0.54 \\ &= 0.46 \end{aligned}$$

The metric is a modelling choice. There is no formula that tells you which distance to use, it follows from understanding your data and the context you are in. In the following scenarios, decide which distance metric is most appropriate and justify your choice. First

reason geometrically, then commit to an answer. Zoom-in so you can not see the margin notes, and try to solve it on your own first. Then read the margin notes for the answer.

A NEWS PLATFORM represents 10 000 articles as word-frequency vectors. Some articles are 100 words long, others 5 000 words. You want to find articles that cover similar topics.

Cosine similarity. Article length inflates the magnitude of the frequency vector but does not change its direction. Cosine similarity ignores magnitude, comparing only the distribution of words.

A DELIVERY COMPANY plans routes through Manhattan's grid. Pickup locations are given as (street, avenue) coordinates.

Manhattan distance. Movement on a grid is constrained to horizontal and vertical steps. The shortest driving path between two intersections follows the L_1 norm, not the straight line.

A COFFEE QUALITY LAB measures sourness (0 to 200) and bitterness (0 to 500) for each batch.

Mahalanobis distance. The features live on different scales and are correlated. Mahalanobis accounts for both by using the covariance matrix, producing ellipsoidal contours that align with the data's spread.

GIVEN THREE COFFEE BATCHES measured by sourness and bitterness:

$$
\begin{aligned}
P_1 &= (80,\ 400) \\
P_2 &= (200,\ 160) \\
P_3 &= (110,\ 220)
\end{aligned}
$$

compute the sample covariance matrix and use it to find the Mahalanobis distance between P_1 and P_3.

The mean vector:

$$
\begin{aligned}
\mu_x &= 130 \\
\mu_y &= 260 \\
\mu &= (130,\ 260)
\end{aligned}
$$

Deviations from the mean:

$$
\begin{aligned}
P_1 - \mu &= (-50,\ \ 140) \\
P_2 - \mu &= (\ \ 70,\ -100) \\
P_3 - \mu &= (-20,\ -40)
\end{aligned}
$$

The sample covariance matrix sums all three outer products:

$$
\begin{aligned}
\Sigma &= \frac{1}{N-1} \sum (x_i - \mu)(x_i - \mu)^T \\
&= \frac{1}{2} \begin{pmatrix} 7800 & -13200 \\ -13200 & 31200 \end{pmatrix} \\
&= \begin{pmatrix} 3900 & -6600 \\ -6600 & 15600 \end{pmatrix}
\end{aligned}
$$

We invert Σ analytically using the 2×2 formula:

$$\begin{aligned}\det(\Sigma) &= 3900 \cdot 15600 - 6600^2 \\ &= 17{,}280{,}000 \\ \Sigma^{-1} &= \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}\end{aligned}$$

For a 2×2 matrix:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

$$A^{-1} = \frac{1}{ad - bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

Swap the diagonal, negate the off-diagonal, divide by the determinant.

The difference vector is $\delta = P_3 - P_1 = (30,\ -180)$. Applying Σ^{-1}:

$$\begin{aligned}\Sigma^{-1}\delta &= \frac{1}{17{,}280{,}000}\begin{pmatrix} 15600 & 6600 \\ 6600 & 3900 \end{pmatrix}\begin{pmatrix} 30 \\ -180 \end{pmatrix} \\ &= \frac{1}{17{,}280{,}000}\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix}\end{aligned}$$

INTUITIVELY, A^{-1} undoes the transformation A: if A stretches and rotates space, A^{-1} maps everything back, with $AA^{-1} = I$. For Σ, the inverse rescales each dimension by its variance and removes correlations, so distances are measured in standard-deviation units.

Taking the dot product with δ gives the squared distance:

$$\begin{aligned}d_M^2 &= \delta^T(\Sigma^{-1}\delta) \\ &= \frac{1}{17{,}280{,}000}(30,\ -180)\begin{pmatrix} -720{,}000 \\ -504{,}000 \end{pmatrix} \\ &= \frac{69{,}120{,}000}{17{,}280{,}000} \\ &= 4 \\ d_M &= \sqrt{4} \\ &= 2\end{aligned}$$

Compare with the Euclidean distance, which does not account for the different scales and correlation. The raw displacement of 30 sourness units and 180 bitterness units yields an overall distance of 182.5, making P_3 seem very far from P_1, driven almost entirely by the bitterness difference:

$$\begin{aligned}d_E(P_1, P_3) &= \sqrt{(110-80)^2 + (220-400)^2} \\ &= \sqrt{900 + 32400} \\ &= \sqrt{33300} \\ &\approx 182.5\end{aligned}$$

The Mahalanobis distance is far smaller because the covariance matrix rescales for the large variance difference between features. The bitterness axis has a much larger spread (σ^2=15600 vs σ^2=3900), so a displacement of 180 bitterness units is less unusual than it appears, and contributes less to the Mahalanobis distance accordingly.

STANDARDIZATION does not change the data, it changes the expanse and spread of the space you measure it in. Units disappear; and make way for a more abstract notion of distance, in terms of how many standard deviations apart the data points are.

$C_1 = (300,\ 7)$
$C_2 = (500,\ 6)$
$C_3 = (700,\ 3)$

daily cups sold, customer rating (1 to 10)

THE EFFECT OF STANDARDIZATION on café data. Three cafés C_1, C_2, C_3 recorded as daily cups sold and customer rating.

Compute the Euclidean distance on raw features, standardize all three points, and recompute. On raw features, daily sales dominate completely; the two pairs involving C_2 look equally close:

$$\begin{aligned} d_E(C_1, C_2) &= \sqrt{(500-300)^2 + (6-7)^2} \\ &= \sqrt{40\,000 + 1} \\ &\approx 200.0 \\ d_E(C_2, C_3) &= \sqrt{(700-500)^2 + (3-6)^2} \\ &= \sqrt{40\,000 + 9} \\ &\approx 200.0 \end{aligned}$$

The rating differences (1 and 9) are drowned out by the cup counts (40,000 each). The means and standard deviations are:

$$\mu_{\text{cups}} = 500, \qquad \sigma_{\text{cups}} = 200$$
$$\mu_{\text{rating}} \approx 5.33, \qquad \sigma_{\text{rating}} \approx 2.08$$

After z-score normalization the cafés map to:

$$C_1' \approx (-1,\ 0.80), \qquad C_2' \approx (0,\ 0.32), \qquad C_3' \approx (1,\ -1.12)$$

THE CUPS standardize cleanly because $300, 500, 700$ are evenly spaced. The ratings do not: $7, 6, 3$ give a non-integer mean and an irrational standard deviation. This is what real data looks like.

Recomputing the distances:

$$\begin{aligned} d(C_1', C_2') &= \sqrt{(0-(-1))^2 + (0.32-0.80)^2} \\ &= \sqrt{1 + 0.23} \\ &\approx 1.11 \\ d(C_2', C_3') &= \sqrt{(1-0)^2 + (-1.12-0.32)^2} \\ &= \sqrt{1 + 2.07} \\ &\approx 1.75 \end{aligned}$$

C_2 is now clearly closer to C_1 than to C_3. In raw space, sales made both pairs look identical; after standardization, the similar reviews of C_1 and C_2 pull them together while C_3's low rating pushes it away.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

20 coffee samples
acidity (pH, 4.0 to 7.0)
bitterness (1 to 10)
brew strength (mg/mL, 5 to 25)

four varieties: cold brew, drip, espresso, latte

A TOY DATASET TO EXPLORE. You are given measurements of 20 coffee samples with three features: *acidity* (pH, 4.0 to 7.0), *bitterness*

(1 to 10), and *brew strength* (mg/mL, 5 to 25). The samples belong to four varieties: cold brew, drip, espresso, and latte. Load the dataset and scatter-plot acidity vs. bitterness. Then compute the Euclidean distance matrix and identify the most similar pair, is the result plausible? Apply z-score normalization, recompute the distance matrix, and observe how the nearest neighbors shift. Compute cosine similarity between all pairs and compare it to the Euclidean grouping. Finally, compute the sample covariance matrix of the normalized data and check whether any features are correlated.

WHAT TO OBSERVE. Without normalization, brew strength dominates similarity because its scale (5 to 25 mg/mL) dwarfs the others. A latte and an espresso with similar strength appear closer than two espressos with the same acidity and bitterness profile. After normalization, all three features contribute equally and the four natural clusters become visible. Cosine similarity groups samples by the *ratio* of their features, which may merge varieties with similar flavor profiles but very different intensities.

Self-Reflection and Recap

SELF-REFLECTION Questions which can guide your thoughts during the exercises and afterwards:

- How do we distinguish between supervised and unsupervised learning?
- Elaborate on the differences between $\tilde{y}$, $\hat{y}$, and y?
- What are the key properties that define a distance metric?
- How do different distance metrics capture different notions of similarity?
- What is a feature space, and how do data points relate to it?
- What do we mean by feature engineering and feature transformation?
- What can happen when features are on different scales or magnitudes?
- How can we engineer and transform feature spaces?
- Why do we transform feature spaces with respect to pattern? And what are the common transformations?

RECAP of Key Concepts:

- Unsupervised learning discovers pattern in unlabeled data
- Distance metrics define what similar means
- Feature spaces can be engineered and transformed to reveal different patterns

SO FAR, WE DID MEASURE SIMILARITY BETWEEN SINGLE DATA POINTS. How do we embed this information across data points? Is it possible to represent pattern in the data as a whole, rather than just pairwise similarities? In the next chapter we explore clusters and clustering algorithms, which are the most classic way to represent pattern in unsupervised learning.

WITHOUT KNOWING ABOUT LABELS OR SEMANTICS we can tell which samples are similar.

TEASER. How do we represent structure beyond pairwise similarities and distances?

FEEDBACK

Centroid Clustering

From Similar to of the Same

The Why

In the previous chapter, we established that distance is a choice that is to be engineered[4]: it encodes what similar means in a given feature space. But similarity is a local, pairwise concept so far.

As a human looking at data, we immediately see clusters C of points that belong together, and we have an intuition for which new points would fit into which cluster. The process of finding them is called clustering.

[4] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. ISBN 978-0387310732. http://research.microsoft.com/~cmbishop/PRML/

A partition or a clustering assigns every point to exactly one cluster, with no cluster left empty. In this lecture we will learn about K-Means[5], which produces a *hard* partition: membership is binary, not graded.

[5] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967. https://scholar.google.com/scholar?q=MacQueen+1967+methods+classification+multivariate+observations

Given that means to encode similarity over whole sets of samples, we can ask a more complex question, one that goes beyond pairs of points. Being able to answer which point belongs to which cluster is a predictive power that allows us to generalize from observed data to new instances.

Binary membership is a strong assumption that later clustering methods will soften.

In order to move from representation to leveraging pattern we have good reason to find ways to,

- compute partitions, of the data, given a distance measure.
- formalize what makes a well-defined partition by specifying an appropriate objective.
- recognize the geometric assumptions and finding ways to represent cluster characteristics.

Figure 10: Clustering allows us to move from a) and b) are similar to a) and b) are of the same. *Image is generated with NanoBanana.*

Hands On Experience

CONSIDER twelve data points in two dimensions. Which points belong together? How many clusters do you see?

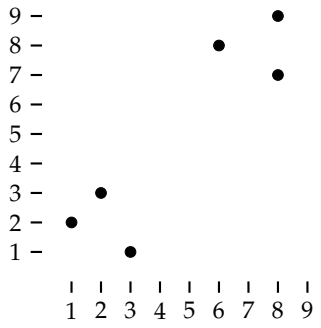


Figure 11: Six points in 2D. How many clusters do you see?

HOW MANY CLUSTERS do you see? Two seems natural, but could you argue for one? For six?

WITHOUT GROUND TRUTH, there is no single right answer, as a context for any answer could be engineered. However, some answers are better than others, and it does not come as a surprise when I tell you that most people see two clusters in this data.

THE FUNDAMENTAL QUESTION this chapter answers is how do we find the clusters mathematically? We will see how K-Means assigns each point to a cluster; and how cluster centroids function as representative points for each cluster.

WHERE WOULD YOU PLACE a representative point for each cluster? Could you think of other measures to characterize and represent clusters, beyond a single point?

A CENTROID c is the mean position of all points assigned to a cluster. It need not be an actual data point; it is the center of mass of the cluster. The centroid exists in the same space as the data. With a different distance, a different notion of center is needed. For sake of simplicity and interpretability, we tie ourselves to Euclidean distance and arithmetic means. But it is worth getting an intuition for other distances as well.

THE LEARNING OBJECTIVES of this lecture:

- Understand the K-Means objective and algorithm, including convergence.
- Apply initialization strategies and K-selection methods.
- Identify when K-Means fails and understand why.

The K-Means Algorithm

A DATA POINT is a vector $x_i \in \mathbb{R}^d$, where d is the number of features and $i = 1, \ldots, n$ indexes the observations.

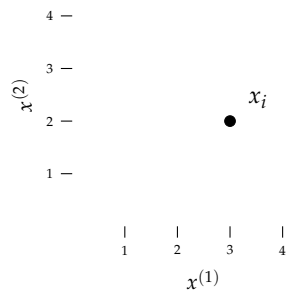


Figure 12: A single data point x_i in $\mathbb{R}^2$.

A CLUSTER is a non-empty subset $C_k \subseteq \{x_1, \ldots, x_n\}$. Each point belongs to exactly one cluster; this is the hard assignment assumption, meaning clusters are disjoint:

$$C_j \cap C_k = \emptyset \quad \text{for } j \neq k$$

A clustering of the data is a collection of K clusters $\{C_1, \ldots, C_K\}$ that is exhaustive, every point is accounted for:

$$\bigcup_{k=1}^{K} C_k = \{x_1, \ldots, x_n\}$$

K-MEANS finds a clustering of n data points into K clusters by minimizing the total squared distance from each point to its cluster's centroid. This quantity is called inertia or within-cluster sum of squares (WCSS):

MINIMIZING J is NP-hard (effort exponential in n and K) in general. K-Means finds a local optimum, not necessarily the global one. The result depends on initialization, and is thus non-deterministic. Shout out numerical methods.

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \tag{7}$$

where the mean $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ is the centroid of cluster C_k.

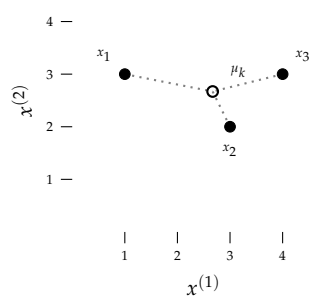


Figure 13: Cluster $C_k = \{x_1, x_2, x_3\}$ with centroid μ_k. Dotted lines show the distances whose squares sum to J.

THE ALGORITHM ALTERNATES BETWEEN TWO STEPS until the assignments no longer change. It remains one of the most cited algorithms in machine learning, in part because it is easy to implement, easy to interpret, and fast enough to run on large datasets.

Algorithm 1: K-Means

Require: Data $X = \{x_1, \ldots, x_n\}$, number of clusters K, initial centroids $\mu_1, \ldots, \mu_K$
Ensure: Clustering $\{C_1, \ldots, C_K\}$, centroids $\{\mu_1, \ldots, \mu_K\}$

```
repeat
    Assignment: c_i ← arg min_k ||x_i − μ_k||^2 for each x_i
    Update: μ_k ← (1/|C_k|) Σ_{x_i : c_i = k} x_i for each k
until assignments no longer change
```

STANDARD ALGORITHMIC FORM was described by Lloyd

S. P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. DOI: 10.1109/TIT.1982.1056489. https://scholar.google.com/scholar?q=Lloyd+1982+least+squares+quantization+PCM

A TRACE THROUGH THE ALGORITHM on three points with $K{=}2$ makes this concrete. With initial centroids at $\mu_1{=}x_1{=}(1,3)$ and $\mu_2{=}x_3{=}(4,3)$:

- For each point, the assignment step asks: which centroid is closest?
- For each cluster, the update step asks: given the current assignments, where is the new centroid?

STEP 1: INITIALIZATION. We place two centroids at $\mu_1 = (1,4)$ and $\mu_2 = (4,1)$, away from any data point. At this moment the positions of the centroids is arbitrary, and the mean variable is not meaningful.

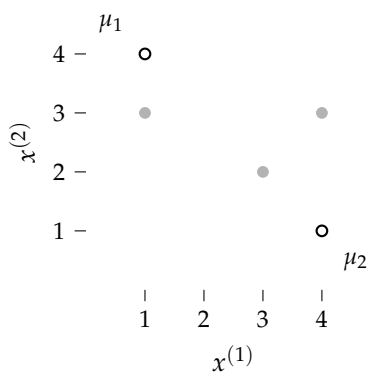


Figure 14: Initialization: two centroids placed away from the data. All points are unassigned.

STEP 2: ASSIGNMENT. Each point joins the cluster of its nearest centroid. For that, we calculate squared distances.

$$
\begin{aligned}
d^2(x_1,\mu_1) &= 0+1 = 1\\
d^2(x_1,\mu_2) &= 9+4\\
&= 13 \;\rightarrow\; C_1\\
d^2(x_2,\mu_1) &= 4+4 = 8\\
d^2(x_2,\mu_2) &= 1+1\\
&= 2 \;\rightarrow\; C_2\\
d^2(x_3,\mu_1) &= 9+1 = 10\\
d^2(x_3,\mu_2) &= 0+4\\
&= 4 \;\rightarrow\; C_2
\end{aligned}
$$

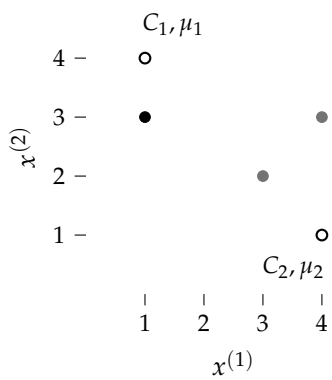


Figure 15: Assignment: $x_1 \rightarrow C_1$ (dark), x_2 and $x_3 \rightarrow C_2$ (grey).

STEP 3: UPDATE. Each centroid moves to the mean of its cluster.

$$
\begin{aligned}
\mu_1' &= x_1\\
&= (1,\ 3)\\
\mu_2' &= \tfrac{1}{2}\big((3,2)+(4,3)\big)\\
&= (3.5,\ 2.5)
\end{aligned}
$$

ON ′ NOTATION. The prime symbol (′) usually denotes updates in iterative algorithms.

CONVERGENCE is guaranteed in finite steps. The argument rests on three observations: first, each assignment step can only decrease

LOCAL OPTIMUM. Convergence does not mean the algorithm found the best clustering. It found a clustering. A different initialization may yield a better one.

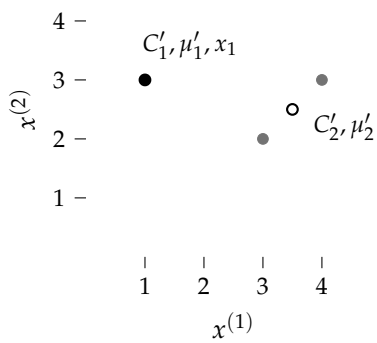


Figure 16: Update: both centroids move. μ_1 coincides with $x_1 = (1, 3)$; μ_2 jumps from $(4, 1)$ to $(3.5, 2.5)$. Rerunning assignment yields the same clusters: converged at J=1.

J, since every point moves to an equal or closer centroid; second, each update step can only decrease J, since the mean minimizes squared distance to its members; third, there are at most K^n possible assignment configurations. Since J is non-increasing and the number of states is finite, the algorithm must reach a fixed assignment and stop.

Initialization Matters

K-Means is sensitive to where centroids start. The algorithm is thus non-deterministic. This is because the objective is non-convex: it has many local minima, and the algorithm may get stuck in one of them. If two initial centroids fall inside the same natural cluster, this cluster might get split. While at the same time, a different cluster might be left without a centroid, getting merged with a nearby cluster instead, which results in a high-inertia J clustering.

Educated InitializationK-Means++ for instance features educated initialization. Centroids seeded onto data points x_i. The first centroid is chosen uniformly at random. Each subsequent centroid is chosen with probability proportional to its squared distance to the nearest already-chosen centroid: $p(x_i) = D(x_i)^2 / \sum_j D(x_j)^2$, where $D(x_i) = \min_{j<k} \|x_i - c_j\|$. Points far from all current centroids are more likely to be chosen next. This gives a theoretical guarantee on the expected inertia relative to the global optimum.

D. Arthur and S. Vassilvitskii. K-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007. https://scholar.google.com/scholar?q=Arthur+Vassilvitskii+2007+k-means+advantages+careful+seeding

Running the same three points with μ_1=$(2, 2)$ and μ_2=$(4, 4)$ shows how a different start leads to a different, worse result.

Multiple restarts. Running K-Means n times with different random initializations, keep the result with the lowest J. Typical default: n=10, but checking the variance or standard deviation of the resulting Inertias and making an informed decision is recommended.

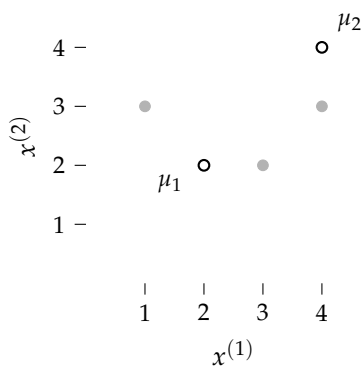


Figure 17: Initialization: centroids at μ_1=$(2,2)$ and μ_2=$(4,4)$. All points unassigned.

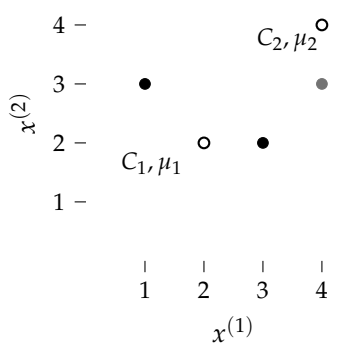


Figure 18: Assignment: x_1 and $x_2 \to C_1$ (dark), $x_3 \to C_2$ (grey).

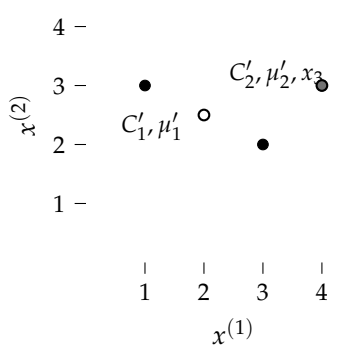


Figure 19: Update: μ_1 moves to $(2, 2.5)$, μ_2 drops to $(4, 3)$. Re-running assignment yields the same clusters: converged at J=2.5, worse than the earlier trace (J=1).

Choosing K

In 2D the number of clusters is often obvious, but in higher dimensions and with more data, or when facing fuzzy clusters or cluster borders, it is not. While the choice for K is often guided by domain knowledge, it is not uncommon to have no such guidance. Then we have to rely on heuristics.

The elbow method plots inertia J against K and looks for a kink: the point where adding one more cluster yields diminishing returns. For our three points, J drops from K=1 to K=2, then reaches zero at K=3.

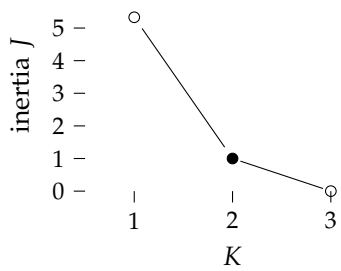


Figure 20: Inertia J as a function of K for the three trace points. The kink at K=2 (filled dot) matches the two natural clusters. K=3 trivially gives J=0 - jap, that is overfitting.

The elbow is not always clear. On real data, the curve may decrease smoothly with no obvious kink. The elbow is a heuristic, not a guarantee.

The silhouette score builds on two distances defined for each point x_i. This allows us to evaluate the clustering on a per-point scale. Think about how interpretable and beautiful that would look when colour-coded.

- **Intra-cluster distance** $a(i)$: mean distance from x_i to all other points in its own cluster. Small $a(i)$ means the point fits tightly with its neighbours.
- **Inter-cluster distance** $b(i)$: mean distance from x_i to all points in the nearest other cluster. Large $b(i)$ means the point is far from the next-best alternative.

A well-placed point has small $a(i)$ and large $b(i)$. The silhouette score combines both into a single value per point:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}$$

$s(i)$ ranges from -1; inter $<$ intra: point fits better in a neighbouring cluster. To $+1$; intra $\ll$ inter: point is tightly placed and well-separated The mean silhouette over all points is a quality score for the full clustering. Choose K that maximizes it.

Simplifications in per-cluster metrics. In practice, $a(i)$ is often computed as the distance to the cluster centroid, rather than the mean distance to all other points. And b(i) is often computed as the distance to the nearest other centroid, rather than the mean distance to all points in the nearest cluster. And even more simplifications are possible, when only calculating the silhouette for the centroids themselves.

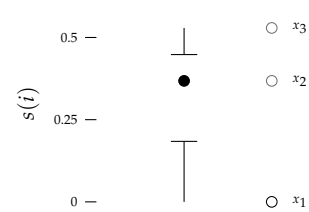
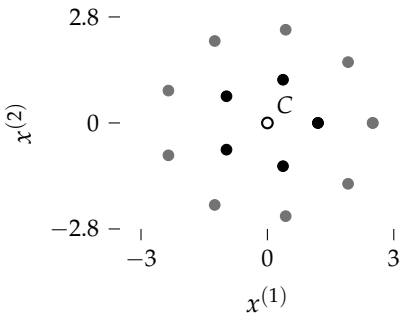


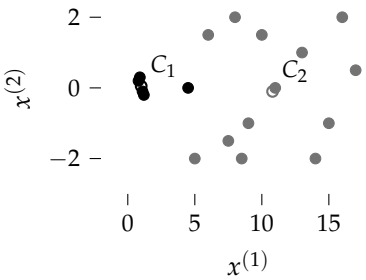
Figure 21: Silhouette scores for K=2. Box plot (left): median ≈0.37, mean ≈0.30, IQR [0.18, 0.45]. Individual scores per point (right, coloured by cluster).

When K-Means Fails

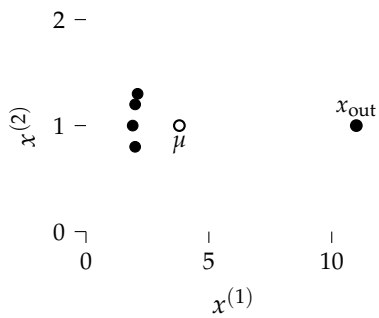
K-Means assumes clusters are roughly spherical, equally sized, and equally dense. Remember that distances measures are a choice.



Non-Convex Shapes
Figure 22: The natural clusters are concentric rings. Both share the same centroid at the origin, so K-Means cannot separate them; it will split the data along a straight boundary through the centre instead.



Cluster Density and Gravity
Figure 23: C_1 has four tightly packed points near $x = 1$; C_2 has thirteen points spread from $x = 5$ to $x = 17$, pulling its centroid to $x \approx 10.8$. The decision boundary falls near $x \approx 6$, so the point at $x = 4.5$ is assigned to C_1 even though it sits at C_2's left edge—a direct consequence of K-Means optimising inertia rather than visual proximity.



Outliers
Figure 24: Sensitivity to outliers: four tight points form a natural cluster near $(2, 1)$, but a single outlier ($\times$) pulls the centroid μ to $(3.8, 1)$, well outside the dense group. K-Means uses the mean, which is not robust to extreme points.

When K-Means struggles, the underlying issue is always the same: the Euclidean distance to a centroid is the wrong measure of belonging for that data. Can you think of another failure mode? Let me know.

K-Medoids uses actual data points as cluster centers instead of means, making it robust to outliers and compatible with non-Euclidean distances.

Mini-Batch K-Means updates centroids on random subsets of the data only, allowing us to scale to millions of points at the cost of a slightly noisier result.

Examples & Exercises

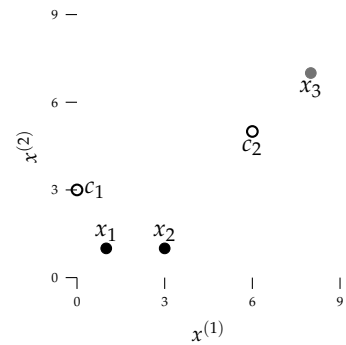


Figure 25: Three coffee samples x_1, x_2, x_3 with initial centroids $\mu_1{=}(0,3)$ and $\mu_2{=}(6,5)$ (circles).

COMPUTING BY HAND builds the intuition that no amount of function calls can replace. Step through the arithmetic slowly. Implementing from scratch comes right after, but let's anchor the mechanics first.

GIVEN THREE POINTS $x_1{=}(1,1)$, $x_2{=}(3,1)$, $x_3{=}(8,7)$, run one full K-Means iteration with $K{=}2$. The initial centroids are $c_1{=}(0,3)$ and $c_2{=}(6,5)$, neither coinciding with a data point. Your tasks: perform the assignment step, the update step, verify convergence, and evaluate the quality of the clustering by computing the inertia.

ASSIGNMENT STEP. For each point compute $d^2(x, c_k) = \sum_j (x_j - c_{kj})^2$ and assign to the nearest centroid. Here is the full trace for point $x_1{=}(1,1)$:

$$\begin{aligned} d^2(x_1,\, c_1) &= (1-0)^2 + (1-3)^2 \\ &= 1 + 4 \\ &= 5 \end{aligned}$$

$$\begin{aligned} d^2(x_1,\, c_2) &= (1-6)^2 + (1-5)^2 \\ &= 25 + 16 \\ &= 41 \end{aligned}$$

Since $5 < 41$, point x_1 is assigned to C_1. Now compute the distances for x_2 and x_3 yourself, then verify against the table below.

Point	$d^2(\cdot, \mu_1)$	$d^2(\cdot, \mu_2)$	**Cluster**
x_1 $(1,1)$	**5**	41	C_1
x_2 $(3,1)$	**13**	25	C_1
x_3 $(8,7)$	80	**8**	C_2

Table 1: Assignment step with $c_1{=}(0,3)$, $c_2{=}(6,5)$. Bold values indicate the minimum in each row; each point is assigned to the corresponding cluster.

UPDATE STEP. Recompute each centroid as the mean of its assigned points. Here is the full calculation for μ_1':

$$\begin{aligned}\mu_1' &= \frac{1}{|C_1|}\sum_{x\in C_1} x \\ &= \tfrac{1}{2}\big((1,1)+(3,1)\big) \\ &= \tfrac{1}{2}(4,\,2) \\ &= (2,\,1)\end{aligned}$$

Now derive μ_2' yourself (C_2 contains only a single point) and then check:

$$\begin{aligned}\mu_2' &= x_3 \\ &= (8,\,7)\end{aligned}$$

CONVERGENCE CHECK. Run the assignment step once more with $\mu_1'{=}(2,1)$ and $\mu_2'{=}(8,7)$: you should recover the same two clusters $C_1{=}\{x_1,x_2\}$, $C_2{=}\{x_3\}$. The algorithm has converged after a single iteration.

The inertia at convergence is:

$$\begin{aligned}J &= \|x_1-\mu_1'\|^2+\|x_2-\mu_1'\|^2+\|x_3-\mu_2'\|^2 \\ &= \big((1{-}2)^2+(1{-}1)^2\big)+\big((3{-}2)^2+(1{-}1)^2\big)+0 \\ &= 1+1+0 \\ &= 2\end{aligned}$$

Calculate the inertia for $K = 1$ yourself. Assume that the single centroid is at the mean of all three points, and is thus quick to compute. What is the Inertia for $K = 3$, is there a need to compute it, and why?

CHOOSING K WITH THE ELBOW METHOD. The table in the margin gives the optimal inertia for $K = 1, 2, 3$ on the same three points. Sketch the inertia curve and identify the elbow: the value of K at which adding another cluster stops yielding a large reduction. Which K do you select, and why?
Note that $K{=}3$ always reaches $J{=}0$ by placing one centroid per point; this tells us nothing about structure in the data.

Inertia values:

K	J
1	50
2	2
3	0

COMPUTE THE SILHOUETTE SCORES (optional) for the three trace points $x_1{=}(1,3)$, $x_2{=}(3,2)$, $x_3{=}(4,3)$ with $K{=}2$, where $C_1{=}\{x_1\}$ and $C_2{=}\{x_2, x_3\}$.

For x_1 (singleton cluster, so $a(1){=}0$ by convention):

$$\begin{aligned} b(1) &= \tfrac{1}{2}\big(d(x_1,x_2) + d(x_1,x_3)\big) \\ &= \tfrac{1}{2}\big(\sqrt{5} + \sqrt{9}\big) \\ &= \tfrac{1}{2}(2.24 + 3) \\ &\approx 2.62 \\ s(1) &= \frac{2.62 - 0}{\max(0,\, 2.62)} \\ &= 0 \end{aligned}$$

For x_2 and x_3 (both in C_2, with $d(x_2,x_3) = \sqrt{2} \approx 1.41$):

$$\begin{aligned} a(2) &= d(x_2,x_3) \approx 1.41 \\ b(2) &= d(x_2,x_1) \\ &= \sqrt{5} \\ &\approx 2.24 \\ s(2) &= \frac{2.24 - 1.41}{2.24} \approx 0.37 \end{aligned}$$

$$\begin{aligned} a(3) &= d(x_3,x_2) \approx 1.41 \\ b(3) &= d(x_3,x_1) \\ &= \sqrt{9} \\ &= 3 \\ s(3) &= \frac{3 - 1.41}{3} \approx 0.53 \end{aligned}$$

The mean silhouette score is $\bar{s} = (0 + 0.37 + 0.53)/3 \approx 0.30$, matching the box plot in the theory section. Note that x_1 drags the mean down: singleton clusters always score $s{=}0$.

A COFFEE AROMA DATASET TO EXPLORE. Load the dataset of 1,797 two-dimensional aroma fingerprints from ten brewing methods and scatter-plot the two aroma coordinates, coloring each sample by its true variety. Implement the assignment step, the update step, and the inertia function from scratch, then combine them into a complete K-Means run. Run K-Means with $K{=}10$ twenty times using different random seeds and compare the resulting inertia values.

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

Then vary K from 1 to 15, record the best inertia across three restarts for each, and plot the elbow curve.

RANDOM SEEDS. Random seeds are integers that initialize the pseudo-random number generator, ensuring reproducibility.

WHAT TO OBSERVE. Inertia varies noticeably across the twenty runs at $K{=}10$: some seeds place two starting centroids inside the same

brewing group and the algorithm never recovers, producing a measurably worse partition. The elbow curve bends near $K=10$, though not sharply, because several brewing varieties overlap in the two-dimensional fingerprint space. Adding clusters beyond the elbow yields diminishing inertia reduction, confirming that the algorithm is splitting existing groups rather than discovering new ones.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the K-Means objective actually minimize, and why is that a reasonable definition of a good clustering?
- Why does K-Means always converge, and why does convergence not guarantee the best solution?
- Which two improvements of K-means did we mention around initialization?
- How can we mitigate the effect of outliers on K-Means?
- When would you use the elbow method? When would you not?
- How does the silhouette score measure something different from inertia?
- What are the three most notorious failures of K-Means, and as a teaser how can we mitigate those?
- Given a new dataset, what would be your first three steps before running K-Means?

RECAP of Key Concepts:

- K-Means minimizes inertia by alternating assignment (points to nearest centroid) and update (centroid to cluster mean)
- Multiple random initializations is a common heuristics to mitigate local minima
- The elbow and silhouette score are complementary heuristics for choosing K
- K-Means assumes spherical, equally sized clusters; it fails on rings, elongated shapes, and density differences

K-MEANS IS FLAWED. Every failure mode in this chapter reduces to the same problem; Euclidean distance to a centroid is the wrong measure of membership for that data. But there is a more structural limitation: K-Means must commit to a single K. With K=3 you see roast regions; with K=10 you see brewing methods. You cannot see both at once, and the elbow heuristic only helps you pick one level and drop the remaining pattern.

TEASER How can we built the full hierarchy of clusterings first, capture the complete pattern across all levels, and allow for post hoc selections?

FEEDBACK

Hierarchical Clustering

All levels at once.

The Why

K-Means left us with three sharp problems.

The centroid assumption forces every cluster to be convex and compact. A single mean position must stand in for all members, so the algorithm breaks whenever clusters are elongated, ring-shaped, or split across multiple densities. All cluster characteristics and structure information are lost and filtered when the mean is computed. The centroid then lands somewhere no data actually lives, and every assignment made from that position is wrong by construction.

Sensitivity to initialization and outliers means results are non-deterministic. A single extreme point can pull a centroid far from the dense mass of its cluster, silently distorting every assignment downstream. Different random starts can converge to entirely different partitions of the same data.

A single scale of structure is all K-Means can see at once. With K=2 you recover two broad groups; with K=10 you recover ten fine subgroups. Both are valid descriptions of the same data at different levels of zoom, yet K-Means forces you to choose one and discard the other. This is not a flaw in the algorithm but a flaw in the question: asking "how many clusters?" presupposes that data has one natural scale.

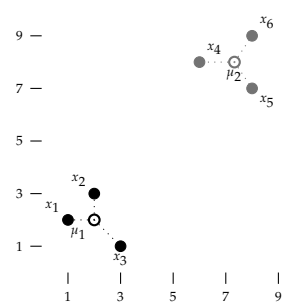


Figure 26: The canonical six points in two natural clusters $K = 2$. Open circles mark the centroids μ_1=(2, 2) and μ_2≈(7.3, 8); dotted lines show the distance each point contributes to inertia.

Hierarchy is the natural answer. Structure in real data rarely lives at a single scale. Biological taxonomy nests species inside genera inside families inside orders. A document corpus organizes words into topics and topics into domains. File systems nest folders inside folders. In each case, the right question is not "how many clusters?" but "which level of abstraction do I need right now?".

HIERARCHICAL CLUSTERING produces a dendrogram: a binary tree in which leaves are individual data points and each internal node records the distance at which two sub-trees were merged. The full range of partitions, from $K{=}1$ to $K{=}n$, lives in this one structure.

IN ORDER TO MOVE FROM A FLAT PARTITION TO A FULL HIERARCHY of clusterings, we have good reason to find ways to,

- extend pairwise point distances to cluster distances, making the notion of closest pair well-defined at every scale.
- record each merge in a tree structure that makes every partition from $K{=}1$ to $K{=}n$ readable in a single pass.
- understand how the choice of cluster distance and linkage design shapes the resulting hierarchy and where it fails.

THE SAME DISTANCE MATRIX that K-Means uses to assign points to centroids can be used to build an entire tree of clusterings. The tree is built in one pass; every value of K is then available for free.

Hands-On Experience

How would you build a hierarchy of clusterings over these six points? Which is the cluster on the highest level of abstraction, K=1? How easy is it to build hierarchy top down? Which are the two clusters on the second highest level, K=2? Still obvious, then it gets trickier when K=3 the first non-trivial design-choices are to be made.

Divisive clustering (top-down) starts with all points in one cluster and recursively splits the cluster with the largest diameter. It is rarely used in practice: splitting is more expensive than merging, and early splits made with global information can be noisier than local merges.

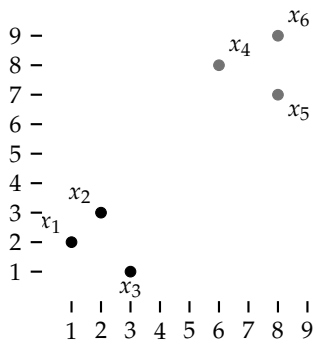


Figure 27: The canonical six points. Which two are closest? Try building a hierarchy top-down, then bottom-up, which one is easier?

Building Hierarchy Bottom Up is more intuitive. Which points are closest to each other? See how this again draws on our understanding of distance as a proxy for similarity? Connect them with a line. Now find the next closest pair and connect them. Keep going until all six points are linked into a tree. At what level of distance would you cut that tree to recover two groups? Three groups?

A horizontal cut at any chosen height yields a flat partition, as we are used to as a result of K-Means.

The fundamental question this chapter answers is how do we formalize the bottom-up merging process? We will see how agglomerative clustering builds a dendrogram from pairwise distances; and how different linkage criteria encode different assumptions about what makes two clusters close.

Linkage, is non-trivial. Two points intuitively merge into a cluster, but beyond that we need a rule for measuring the distance from that cluster to everything else. Do we use the closest pair of points across the two clusters? The furthest? The average? Each answer is a different linkage criterion, and each one tells a different story about what "nearby clusters" means.

The Learning Objectives of this lecture:

- Understand how agglomerative clustering builds a dendrogram through successive minimum-distance merges, and read merge-height gaps to identify the natural number of clusters.
- Apply single, complete, and Ward's linkage and explain how each criterion shapes the resulting hierarchy.
- Identify when hierarchical clustering succeeds where K-Means fails, recognise its failure modes, and explain why scalability requires approximations such as BIRCH.

The Hierarchical Clustering Algorithm

THE AGGLOMERATIVE STRATEGY[6] is bottom-up: given data $X = \{x_1, \ldots, x_n\}$, initialize n singleton clusters $C_i = \{x_i\}$ in an active set and precompute a distance matrix $D[i,j] = d(x_i, x_j)$. At each step the closest pair $(i^*, j^*) = \arg\min_{i \neq j} D[i,j]$ is merged into $C_m = C_{i^*} \cup C_{j^*}$; the distances to remaining active clusters are updated via linkage criterion L. Every merge is recorded as an internal node of the dendrogram T, labeled with the distance $D[i^*, j^*]$ at which it occurred.

[6] J. H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963. DOI: 10.1080/01621459.1963.10500845. https://scholar.google.com/scholar?q=Ward+1963+hierarchical+grouping+optimize+objective+function

THE RESULT T is a binary tree. Every valid flat partition from K=1 to K=n can be read off by making a horizontal cut at the desired height. The $n-1$ merge heights span the full range from the smallest pairwise distance to the diameter of the data.

Algorithm 2: Agglomerative Hierarchical Clustering.

Require: Data $X = \{x_1, \ldots, x_n\}$, linkage criterion L
Ensure: Dendrogram T
Initialize: $C_i \leftarrow \{x_i\}$ for $i = 1, \ldots, n$; active $\leftarrow \{C_1, \ldots, C_n\}$
Compute $D[i,j] \leftarrow d(x_i, x_j)$ for all $i \neq j$
while $|\text{active}| > 1$ **do**
  $(i^*,\ j^*) \leftarrow \arg\min_{i \neq j}\ D[i,j]$
  Record merge: $(C_{i^*},\ C_{j^*})$ at height $D[i^*, j^*]$ in T
  $C_m \leftarrow C_{i^*} \cup C_{j^*}$
  Update $D[m,\ k] \leftarrow L(C_m,\ C_k)$ for each active $C_k \neq C_{i^*}, C_{j^*}$
  Remove C_{i^*} and C_{j^*} from active; add C_m
end while

NOTICE THAT L IS THE ONLY DESIGN CHOICE. The algorithm is otherwise fully deterministic.

INITIALIZATION IS DETERMINISTIC. Each point starts as its own singleton cluster, there is nothing to choose. This stands in sharp contrast to k-means, where a random centroid draw forces multiple restarts for optimization. Here, the starting state is uniquely determined by the data, so two runs on the same dataset always produce the same dendrogram.

TRACE THROUGH OUR SIX POINTS. Dotted edges represent pairwise distance calculations; the dashed edge marks the current merge; gray edges record prior ones.

REMOVE TWO. ADD ONE. In every iteration reduces |active| by exactly one (two clusters are removed and one merged cluster is added)

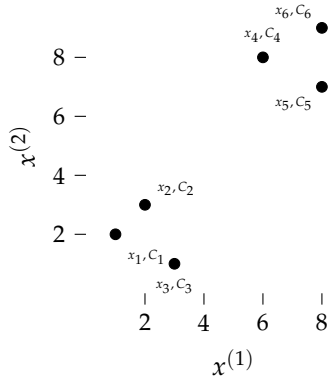


Figure 28:

INITIALIZATION: n=6 singleton clusters C_i=$\{x_i\}$ placed in the active set.

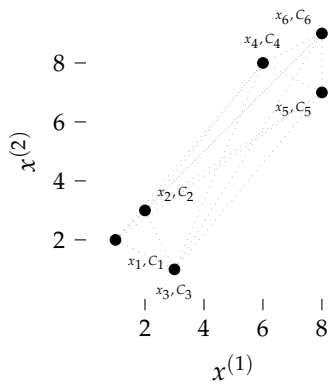


Figure 29:

COMPUTE $D[i,j]$=$d(x_i, x_j)$ for all $\binom{6}{2}$=15 pairs. Each dotted edge is one entry in the distance matrix. At every iteration the algorithm selects the globally shortest edge, $(i^*, j^*) = \arg\min_{i \neq j} D[i,j]$, where i, j range only over the current active set. Once C_{i^*} and C_{j^*} are merged into C_m, they are removed from active and replaced by C_m; past merges are never revisited.

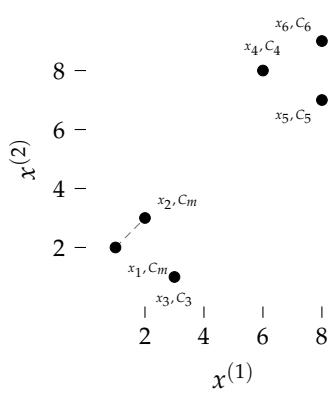


Figure 30:

MERGE 1: the globally smallest distance is $d(x_1, x_2)$. Singletons $\{x_1\}$ and $\{x_2\}$ merge into C_m=$\{x_1, x_2\}$, recorded as an internal node in T with both singletons as children, labeled by $d(x_1, x_2)$: T += $C_7 = \big(C_1,\, C_2,\, d(x_1, x_2)\big)$.

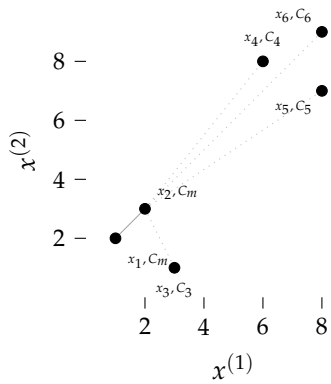


Figure 31:

UPDATE D: after forming $C_7 = \{x_1, x_2\}$, four rows and columns of D are refreshed via $D[7,k] \leftarrow L(C_7, C_k)$. Under single linkage L takes the nearest member: $D[7,k] = \min_{x \in C_7} d(x, C_k)$. Here x_2 is the nearest member of C_7 to all remaining singletons, so every new entry anchors at x_2 (dotted edges).

DISCUSS LINKAGE

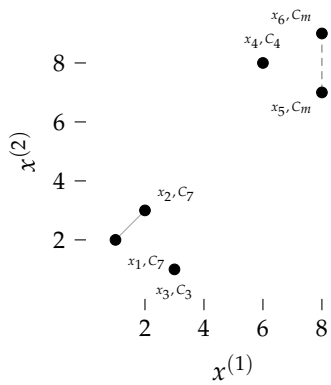


Figure 32:

MERGE 2: the next minimum in the updated D is $d(x_5, x_6)$. Singletons $\{x_5\}$ and $\{x_6\}$ merge into $C_m = \{x_5, x_6\}$, recorded in T as $T \mathrel{+}= C_8 = \big(C_5,\, C_6,\, d(x_5, x_6)\big)$.

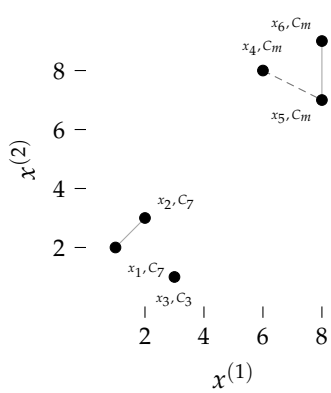


Figure 33:

MERGE 3: $D\big(C_8, x_4\big)$ is minimal under single linkage (distances to a merged cluster use the nearest member). $D\big(C_7, x_3\big)$ ties but x_4 merges first by index. Cluster C_8 absorbs x_4, linking via x_5: $T \mathrel{+}= C_9 = \big(C_8,\, C_4,\, D(C_8, C_4)\big)$.

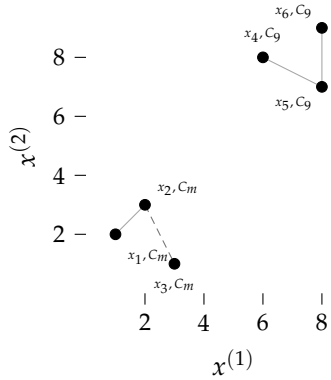


Figure 34:

MERGE 4: also at height $\sqrt{5}$, $D(C_7, x_3) = \sqrt{5}$. Cluster C_7 absorbs x_3, linking via x_2: $T \mathrel{+}= C_{10} = (C_7,\, C_3,\, D(C_7, C_3))$.

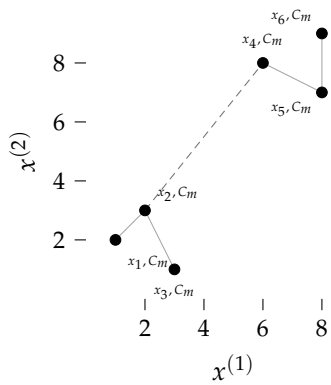


Figure 35:

MERGE 5: $C_{10}=\{x_1, x_2, x_3\}$ and $C_9=\{x_4, x_5, x_6\}$ merge at height $d(x_2, x_4)$, the nearest cross-cluster pair. All six points form one cluster; the dendrogram T is complete.

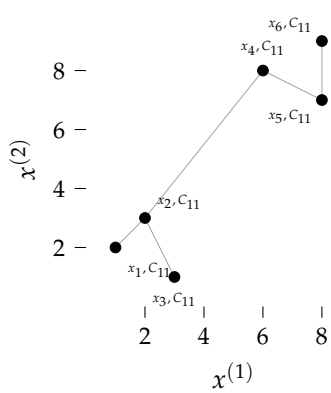


Figure 36:

TERMINATION: $|\text{active}|=1$. All $n=6$ points belong to C_{11}; the five merge edges span the complete single-linkage dendrogram. No further merges are possible.

TERMINATION. The algorithm halts when $|\text{active}| = 1$: exactly one cluster remains, containing all n points. Termination is guaranteed after exactly $n - 1$ merges. At that point T holds $n - 1$ internal nodes, one per merge, and is the complete dendrogram.

LINKAGE CRITERIA BEYOND TWO POINTS IS NON-TRIVIAL determine how the distance between two clusters is computed from the distances between their individual members. The choice of linkage profoundly shapes the dendrogram. Swap L and you get a fundamentally different clustering.

ANSWERS FROM WHERE WE MEASURE THE DISTANCE. The distance then still decides what we are going to merge.

SINGLE LINKAGE

$$D_{\text{single}}(C_i, C_j) = \min_{x \in C_i,\, y \in C_j} d(x, y)$$

The distance between two clusters is the distance between their nearest members. Single linkage admits an implementation based on Prim's minimal spanning tree (details later), which is fast. This also lets it detect elongated, non-convex clusters. This behavior is called chaining, and is also a weakness; a single close pair of points can bridge two otherwise distant clusters, pulling them into one.

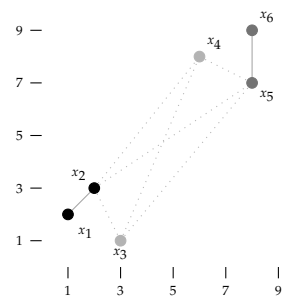


Figure 37: Single linkage: dotted lines show $d_{\min}$ for each of the six active cluster pairs. The smallest, $d(x_2, x_3) = d(x_4, x_5)$, determines the next merge.

COMPLETE LINKAGE

$$D_{\text{complete}}(C_i, C_j) = \max_{x \in C_i,\, y \in C_j} d(x, y)$$

The distance between two clusters is the distance between their furthest members. Complete linkage tends to produce compact, roughly equal-diameter clusters and is robust to chaining. It may, however, split naturally elongated shapes.

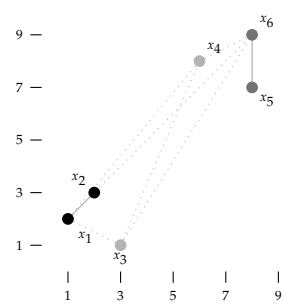


Figure 38: Complete linkage: dotted lines show $d_{\max}$ for each active cluster pair. The largest determines which merge is most expensive.

AVERAGE LINKAGE

$$D_{\text{average}}(C_i, C_j) = \frac{1}{|C_i|\,|C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

Also known as the Unweighted Pair Group Method with Arithmetic Mean. Is a compromise: less prone to chaining than single linkage, less aggressive at squashing elongated clusters than complete linkage. Average linkage averages all cross-cluster pairwise distances.

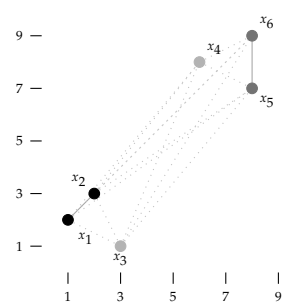


Figure 39: Average linkage: D averages all pairwise cross-cluster distances. Every dotted line contributes to the mean.

Ward's Method

Ward's criterion merges the two clusters whose union minimises the increase in total within-cluster variance:

$$D_{\text{Ward}}(C_i, C_j) = \frac{|C_i|\,|C_j|}{|C_i| + |C_j|}\,\|\mu_i - \mu_j\|^2$$

where μ_i and μ_j are the centroids of C_i and C_j. Ward's method produces compact, spherical clusters and is closely related to K-Means as both minimise within-cluster variance or inertia. Of course this makes Ward share the same weaknesses as K-Means.

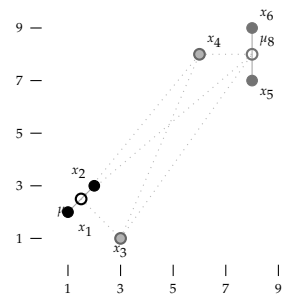
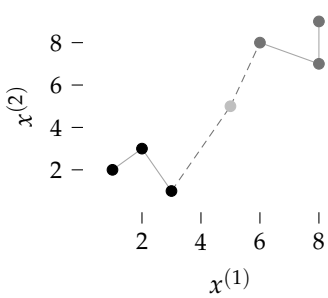


Figure 40: Ward's linkage: dotted lines connect centroids of all active cluster pairs; D is proportional to $\|\mu_i - \mu_j\|^2$ weighted by cluster size.

When Hierarchical Clustering Fails

the root cause is that the linkage criterions optimises locally with no global view nor understanding for scale. Every point gets clustered at some point in the agglomeration.



Chaining and bridging

Figure 41: Single-linkage chaining: one intermediate point (light) bridges two well-separated clusters. Within-cluster merges (solid) complete first; single linkage then greedily closes through the bridge (dashed), incorrectly joining both clusters into one.

Shape Bias Ward's linkage inherits K-Means's spherical bias exactly. The difference is recovery: K-Means re-assigns points each iteration and can partially correct a bad partition. Ward's cannot; the bias compounds irreversibly.

Irreversible greedy merges and local optima. At each step the algorithm selects the globally optimal merge under the current distance matrix: no local minimum trap exists within a single iteration. However, the greedy (no-reassignment) strategy is myopic. Once C_{i^*} and C_{j^*} are merged into C_m, that decision is permanent; no subsequent step can reverse it. A suboptimal early merge propagates through all future distance updates via L, and the resulting dendrogram may differ substantially from the globally optimal hierarchy.

Irrecoverability is what makes chaining and shape bias genuinely dangerous. A bad merge silently propagates through every subsequent distance update; the only remedy is to restart from scratch.

COMPUTATIONAL COMPLEXITY Memory is the hard wall. The distance matrix $D_{i,j}$ requires $O(n^2)$ space. At n=100,000 that is already ≈40 GB for float32: you either fit it in RAM or the algorithm cannot run.

$D_{i,j}$ has $\frac{n(n-1)}{2}$ entries; $O(n^2)$ space.

FLOAT32 AT SCALE. n=10,000 needs ≈400 MB; n=100,000 needs ≈40 GB.

COMPUTE IS THE SOFT WALL. Even when the matrix fits, filling it costs $O(n^2)$. The merge steps then scan and update distances after each of the $n-1$ merges, adding $O(n^2)$ for optimised linkages or $O(n^3)$ for naive implementations.

Algorithm	Time	Notes
K-Means (baseline)	$O(n)$	K, T, d treated as constants
Naive agglomerative	$O(n^3)$	Recomputes all distances each step
Single linkage (SLINK)	$O(n^2)$	Optimal algorithm for single linkage
Average linkage	$O(n^3)$	Without acceleration

Table 2: Complexity of hierarchical clustering variants; K-Means is the linear baseline (K, T, d constant). Naive agglomerative is $O(n^3)$: it rescans the full distance matrix at each of the $n-1$ merge steps.

SINGLE LINKAGE ACHIEVES $O(n^2)$ because its update rule is a simple minimum, as the minimal distance to a merged cluster is the minimum of the distances to its two constituent, deprecated clusters:

$$d(C_i \cup C_j,\, C_k) = \min\big(d(C_i, C_k),\, d(C_j, C_k)\big)$$

SLINK[7] exploits this as each new point requires only a single scan of the current distances; $O(n)$ per update step over $n-1$ merges gives $O(n^2)$ total time.

[7] R. Sibson. SLINK: An optimally efficient algorithm for the single-link cluster method. *The Computer Journal*, 16(1):30–34, 1973. DOI: 10.1093/comjnl/16.1.30. https://scholar.google.com/scholar?q=Sibson+1973+SLINK+optimally+efficient+single-link+cluster

SCALABILITY BY COMPRESSION is regained by BIRCH[8], which breaks the $O(n^2)$ memory barrier by compressing data into a compressed tree in one pass, then running agglomeration on the leaf nodes rather than individual points reducing cost. A threshold T controls compression: A small T produces many fine-grained leaves (accurate but larger tree); a large T merges more points per leaf (faster but coarser). Setting T too large can merge points from different true clusters before agglomeration begins.

[8] T. Zhang, R. Ramakrishnan, and M. Livny. BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pages 103–114, 1996. DOI: 10.1145/233269.233324. https://scholar.google.com/scholar?q=Zhang+Ramakrishnan+Livny+1996+BIRCH+efficient+data+clustering

DENDROGRAM VOCABULARY borrows from genealogy and graph theory. The dendrogram T is a binary tree with n leaves and $n-1$ internal nodes. Each data point x_i is a leaf ℓ_i at height $h=0$. Each internal node v_k records a merge of two children v_a, v_b at height $h_k=D[i^*, j^*]$; the merged cluster is $C_k=C_a \cup C_b$. The root v_{n-1} sits at the highest merge height h_{n-1} and contains all n points. Following the tree upward from any node gives its ancestors; following downward gives its descendants.

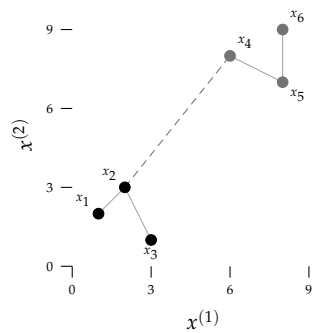
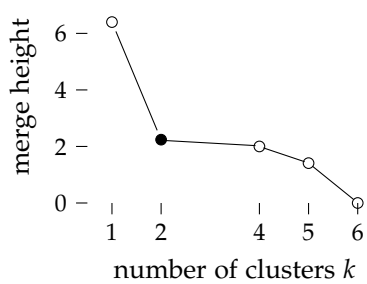


Figure 42: Six points displaying the cut cluster link executed in the dendrograms.

READING A DENDROGRAM requires only one rule: a horizontal cut at height h partitions the data into as many clusters as there are vertical stems crossing that line. Cutting above h_4=6.40 gives one cluster; cutting between h_3=2.24 and h_4=6.40 gives two; cutting between h_1=1.41 and h_2=2.00 gives four; cutting below h_1 gives n singletons. A large gap between consecutive merge heights is the dendrogram equivalent of the elbow: it suggests a natural number of clusters. The kink at k=2 (filled dot) marks the jump from 2.24 to 6.40, confirming two natural clusters. Notice the missing value at $k = 3$.



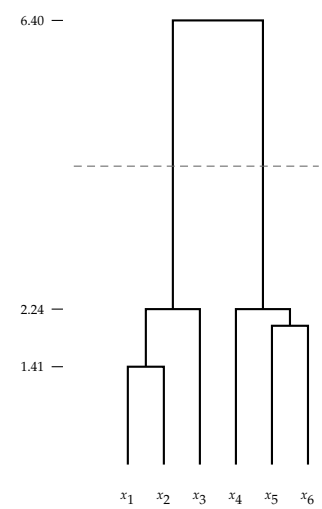


Figure 43: Single-linkage dendrogram on true scale. Heights are Euclidean distances. The jump from 2.24 to 6.40 (dashed cut) yields $C_1=\{x_1, x_2, x_3\}$, $C_2=\{x_4, x_5, x_6\}$.

WARD'S DENDROGRAM for the same six points is shown alongside. Heights are no longer Euclidean distances but variance increases: each merge height records the rise in total within-cluster variance caused by that merge.

COMPARE THE GAPS. Single linkage gave a gap from 2.24 to 6.40 (ratio ≈2.9×). Ward's gives a gap from 3.00 to 96.6 (ratio ≈32×). Ward's tends to amplify the gap signal, making cluster number selection more pronounced.

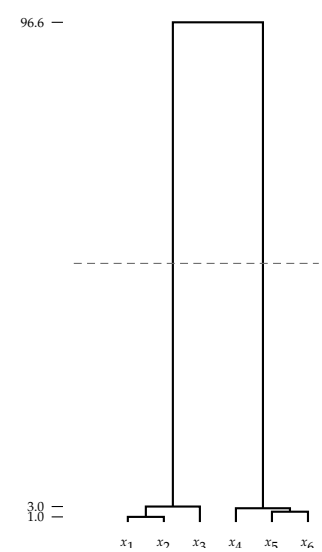


Figure 44: Ward's dendrogram on true scale (Δvar). Four merges pack below 3.0; the final merge towers at 96.6. The gap dwarfs the one in the single-linkage dendrogram above.

How do we infer new data points? The dendrogram is a static structure built on the original data; it does not directly provide a way to assign new points to clusters. But, given a cut of the dendrogram into K clusters, we can assign a new point x_{new} to the cluster whose centroid is closest under the same distance metric used for clustering. Or we assign it to a cluster based on the closest leaf.

Examples & Exercises

GIVEN THREE POINTS x_1=(1, 1), x_2=(3, 1), x_3=(8, 7), run single-linkage agglomerative clustering step by step. These are the same points you clustered with K-Means already, but this time there is no centroid initialization, no random seed, no choice of K. Your tasks: compute the initial pairwise distance table, identify the minimum pair at each step, record each merge, update distances using single linkage, and construct the dendrogram.

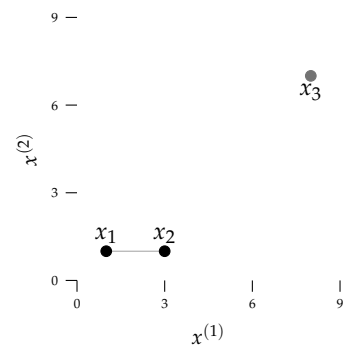


Figure 45: The same three points from the K-Means exercise. No centroids this time; the algorithm needs only pairwise distances.

REUSE BY DESIGN. Working on the same three points lets you compare the two algorithms directly: K-Means needed an initialization and an iterative loop; hierarchical clustering needs neither.

INITIAL DISTANCES. For each pair (x_i, x_j) compute the euclidean distance. With n=3 there are only $\binom{3}{2}$=3 pairs. Here is the full calculation for $d(x_1, x_2)$:

$$\begin{aligned} d(x_1, x_2) &= \sqrt{(3-1)^2 + (1-1)^2} \\ &= \sqrt{4+0} \\ &= 2 \end{aligned}$$

Compute the remaining two distances yourself, then verify against the table below. We use symmetries to reduce compute, so only the upper triangle of the distance matrix is filled; the diagonal is empty since $d(x_i, x_i) = 0$ is trivial and not needed for merges.

	x_1	x_2	x_3
x_1	–	**2.00**	8.49
x_2	–	–	7.81
x_3	–	–	–

Table 3: Initial pairwise Euclidean distances. The global minimum $d(x_1, x_2)$=2 (bold) triggers the first merge.

FIRST MERGE. The minimum entry is $d(x_1, x_2)$=2. Merge x_1 and x_2 into cluster $\{x_1, x_2\}$ at height 2. Update the distance to the remaining point using single linkage. Remember that you do not have to recompute the full distance matrix; only the distances to the new cluster $\{x_1, x_2\}$, repurposing the already computed distances to x_1 and x_2 and solving for the minimum:

$$\begin{aligned} d(\{x_1, x_2\},\ x_3) &= \min(d(x_1, x_3),\ d(x_2, x_3)) \\ &= \min(8.49,\ 7.81) \\ &= 7.81 \end{aligned}$$

SECOND MERGE. Only two active clusters remain: $\{x_1, x_2\}$ and $\{x_3\}$. Their distance is 7.81, so they merge at height 7.81. The dendrogram is complete.

Step	Clusters merged	Height	Updated distances (single linkage)
1	$x_1,\ x_2$	2.00	$d(\{x_1, x_2\},\ x_3)$=7.81
2	$\{x_1, x_2\},\ x_3$	7.81	Final merge; algorithm complete

Table 4: Single-linkage merge history for x_1, x_2, x_3. The gap from 2.00 to 7.81 confirms two natural groups, matching the K-Means result from Chapter 2.

GUARANTEED TERMINATION. How many merges did you just perform? Could there have been more? Fewer? Why?

EACH MERGE REMOVES TWO CLUSTERS from the active set and adds one, reducing |active| by exactly one. Starting from n singletons, the algorithm reaches |active|=1 after exactly $n-1$ steps.

- For n=3: exactly 2 merges.
- For n=1,000: exactly 999 merges.

This also means the dendrogram always has n leaves and $n-1$ internal nodes. Verify this on your two-merge trace: 3 leaves, 2 internal nodes, one complete binary tree T.

CONTRAST WITH K-MEANS. K-Means has no guaranteed iteration count. It can converge in 2 iterations or 200, depending on initialization. Hierarchical clustering always finishes in $n-1$ steps, with no restarts and no randomness.

READING THE DENDROGRAM. Sketch the dendrogram for the three-point trace. It has two horizontal bars: one at height 2.00 (merging x_1, x_2) and one at height 7.81 (merging everything). Now answer the following:

- Cut at h=5. How many clusters do you get? Which points are in each?
- Cut at h=1. How many clusters? What does this tell you?
- What is the largest gap between consecutive merge heights? What K does it suggest?
- How does the spread of the nodes in the dendrogram relate to an elbow plot over K?

LEAF ORDERING TRAP. A dendrogram with n leaves has 2^{n-1} valid leaf orderings: at each internal node you can swap left and right children without changing the hierarchy. Two points adjacent in the plot are not necessarily similar; only the merge height tells you when they joined.

THE GAP is $7.81 - 2.00 = 5.81$: the final merge costs nearly four times as much as the first. A cut anywhere in this gap yields K=2 with C_1=$\{x_1, x_2\}$, C_2=$\{x_3\}$, exactly the partition K-Means found.

LINKAGE COMPARISON WITH CHAINING. Add a bridge point $x_4{=}(5,4)$ to the three points. This point sits between the two natural groups in a region where no cluster really lives.

Compute the initial $\binom{4}{2}{=}6$ pairwise distances. Then trace single linkage to completion. Based on our previous trace, reuse the distances you already calculated, prior to the merge of x_1, x_2 at height 2.00 and calculate the missing distances to x_4. You should find the second merge of $\{x_1, x_2\}, x_4$ at height ≈ 3.61. Finally merge $\{x_1, x_2, x_4\}, x_3$ at height ≈ 4.24

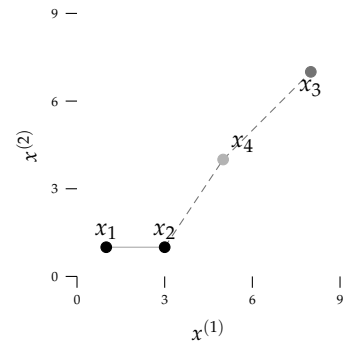


Figure 46: Bridge point x_4 in the gap between the two natural groups. Dashed lines show the chain that single linkage will follow.

THE BRIDGE POINT chains the two groups together through nearest-neighbour links: $x_2 \to x_4 \to x_3$. What happens with our gap signal? The first merge is still x_1, x_2 at height 2.00, but the second merge is now $\{x_1, x_2\}, x_4$ at height 3.61 deluting our signal, and the final merge is $\{x_1, x_2, x_4\}, x_3$ at height 4.24.

NOW REPEAT WITH COMPLETE LINKAGE. Replace each min in the update rule with max. After step 1 (still x_1, x_2 at height 2.00), the updated distance from $\{x_1, x_2\}$ to x_4 under complete linkage becomes:

$$\begin{aligned} d_{\text{comp}}(\{x_1, x_2\},\ x_4) &= \max(d(x_1, x_4),\ d(x_2, x_4)) \\ &= \max(5.00,\ 3.61) \\ &= 5.00 \end{aligned}$$

Check whether x_4 still merges with $\{x_1, x_2\}$ next, or whether a different pair is closer. Trace the remaining merges and compare the complete-linkage dendrogram to the single-linkage one. Which linkage is more vulnerable to the bridge point? Why?

WHY SINGLE LINKAGE IS FAST. Look back at the distance update you performed in the single-linkage trace. When C_i and C_j merge into C_m, the new distance to any remaining cluster C_k is:

$$D_{\text{single}}(C_m,\, C_k) = \min\big(D(C_i,\, C_k),\ D(C_j,\, C_k)\big)$$

One comparison, two table lookups, no access to the original data points. The update depends only on values already in the distance matrix. This means the entire algorithm can run on the $n \times n$ matrix alone; once initialized, the raw data is never touched again.

SLINK exploits exactly this property. Because single linkage only needs nearest-neighbour information, the merge sequence is equivalent to a minimum spanning tree. Prim's algorithm builds that tree in $O(n^2)$, matching SLINK's optimal complexity.

NOW COMPARE WITH WARD'S UPDATE. Recall that Ward's distance measures the increase in total within-cluster variance caused by a merge. For two clusters C_a and C_b with centroids μ_a, μ_b and sizes n_a, n_b:

$$D_{\text{Ward}}(C_a, C_b) = \frac{n_a \cdot n_b}{n_a + n_b} \|\mu_a - \mu_b\|^2$$

To update the distance after a merge, you must recompute the centroid of the merged cluster and then apply the definition again. This means going back to the raw data, unlike single linkage which never looks beyond the distance matrix.

SHORTCUT. The Lance-Williams formula expresses Ward's update purely in terms of existing table entries and cluster sizes, avoiding the detour through centroids. The merge loop still runs in $O(n^2 \log n)$ at best and $O(n^3)$ in the naive implementation, compared to $O(n^2)$ for single linkage.

VERIFY THE DIFFERENCE. For the three-point trace, compute the Ward distance update for $D_{\text{Ward}}(\{x_1, x_2\}, x_3)$ from the definition and confirm that it gives a different value than the single-linkage update. Ward's initial distance between two singletons reduces to $\frac{1}{2}\|x_i - x_j\|^2$. The first merge is still x_1, x_2 (at Ward height $\frac{1}{2} \cdot 4 = 2.00$). After merging, the centroid of $\{x_1, x_2\}$ is $\mu_{12} = (2, 1)$. Now apply the definition to compute the distance to the remaining singleton x_3:

$$\begin{aligned}
D_{\text{Ward}}(\{x_1, x_2\},\ x_3) &= \frac{n_{12} \cdot n_3}{n_{12} + n_3} \|\mu_{12} - x_3\|^2 \\
&= \frac{2 \cdot 1}{2 + 1} \left\| \begin{pmatrix} 2 \\ 1 \end{pmatrix} - \begin{pmatrix} 8 \\ 7 \end{pmatrix} \right\|^2 \\
&= \frac{2}{3} \left\| \begin{pmatrix} -6 \\ -6 \end{pmatrix} \right\|^2 \\
&= \frac{2}{3}\left((-6)^2 + (-6)^2\right) \\
&= \frac{2}{3}(36 + 36) \\
&= \frac{2}{3} \cdot 72 \\
&= 48.00
\end{aligned}$$

Single linkage gave 7.81 (a Euclidean distance); Ward's gives 48.00 (a variance increase). The numbers are not comparable across linkage criteria, but within each criterion they determine the merge order.

BACK-OF-ENVELOPE: SCALABILITY. The distance matrix stores $\binom{n}{2}$ entries. For n=50,000 with 64-bit floats, that is $\binom{50000}{2} \times 8 \approx 9.3$ GB. For n=100,000: roughly 37 GB.

BACK AT THE COFFEETABLE. Same 1 797 coffee aroma fingerprints, ten brewing methods, two coordinates. Compute the pairwise distance matrix, implement single, complete, and average linkage from

CODE will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

scratch, then run all four methods and compare the truncated dendrograms. Cut each tree at $K=3$, colour the scatter plot, and compare with K-Means at $K=3$.

WHAT TO OBSERVE. Single linkage chains into one dominant cluster. Complete and Ward yield balanced partitions; the K-Means result most closely resembles Ward's, because both favour compact, spherical clusters. One hierarchical run gives you every K from 1 to n.

SELF-REFLECTION AND RECAP

SELF-REFLECTION questions to guide your thinking:

- What does the merge height in a dendrogram represent, and why does a large gap between consecutive heights signal a natural number of clusters?
- Why does single linkage produce chaining while complete linkage does not? What property of each formula causes this difference?
- In what situations would you prefer Ward's linkage over single or complete linkage? When would Ward's fail?
- You compute a dendrogram and find that the top-level merge height is only marginally larger than the second-to-top merge height. What does this tell you about the cluster structure?
- What is the computational bottleneck of hierarchical clustering, and what approximate methods exist to overcome it at scale?
- Why is a bad early merge more damaging in hierarchical clustering than a bad initial assignment in K-Means? What mechanism does K-Means have that the dendrogram lacks?

RECAP of Key Concepts:

- Agglomerative clustering greedily merges the two closest clusters at each step, producing a dendrogram that encodes all flat partitions from $K=1$ to $K=n$ in a single structure
- The algorithm terminates in exactly $n-1$ merges; no initialization, no restarts, no convergence check
- Ward's linkage minimises the same within-cluster variance as K-Means; a single Ward dendrogram encodes all partitions from $K=1$ to $K=n$, each equivalent to a K-Means run for that K but without

iterative re-assignment, so Ward's inherits the spherical bias and cannot correct it

- Single linkage detects elongated clusters but suffers from chaining; complete and Ward's linkage produce compact clusters and are more robust to sparse bridge points
- Single linkage is uniquely fast ($O(n^2)$ via SLINK) because its update rule reduces to a single min over existing table entries, equivalent to a minimum spanning tree
- A large gap between consecutive merge heights is the dendrogram signal for a natural number of clusters, analogous to the elbow in K-Means inertia curves
- Leaf adjacency in a dendrogram does not imply similarity; only the merge height matters
- The $O(n^2)$ distance matrix limits hierarchical clustering to moderate-sized datasets without approximation methods such as BIRCH

THE ROOT OF HIERARCHICAL FAILURE. Every linkage criterion measures distance between points or boundaries locally. No criterion has a concept of empty space or of sparsity. In the next Chapter we will introduce and leverage the concept of density. Starting from the single question that hierarchical clustering cannot answer; what are the characteristics of a space or neighbourhood?

TEASER How can we define a cluster without any centroid, any linkage criterion, or any choice of K, label outliers as noise automatically, and still recover clusters of arbitrary shape?

FEEDBACK

Density and Graph-Based Clustering

It is all about that space, about that space.

The Why

Hierarchical clustering gave us every scale of structure at once, but it never learned to say "this point belongs to nothing." Every point, no matter how isolated, is absorbed into some cluster at the final merge. Real data contains noise: GPS measurements with dropouts, sensor readings during interference, survey responses that are genuine outliers. A method that must absorb every point will misrepresent noise as structure.

As we look at a map of taxi pickup locations, we immediately see dense corridors of activity and a handful of isolated points far from any group. Our eye does not force those outliers into the nearest cluster; it marks them as noise and focuses on where the true structure lies. The process of finding regions of density is called density-based clustering.

Density-based clustering reframes the question entirely. Instead of asking how far a point is from a centroid or a neighbouring cluster, it asks how densely packed the neighbourhood around a point is. A cluster is a maximal connected region where points are dense; any point in a sparse region is declared noise outright. Road networks cluster into corridors of taxi GPS traces, not spheres. Social communities cluster into groups of densely connected users, not groups close to a central member.

The key insight is that a cluster need not have a centre. It needs only a notion of neighbourhood: two points belong to the same cluster if they are close enough, and that closeness can chain through intermediate points. This lets clusters take arbitrary shapes, detect noise naturally, and avoid committing to K in advance.

In order to move from pairwise distances to density-aware

Two complementary ideas. Density-based methods (DBSCAN, OPTICS) ask: is this region of space dense enough to be a cluster? Graph-based methods (spectral clustering) ask: is this set of points more connected to each other than to the rest? Both escape the spherical cluster assumption that limits K-Means and hierarchical clustering.

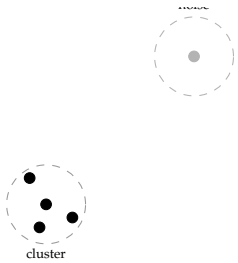


Figure 47: A dense cluster (left) where

CLUSTERS we have good reason to find ways to,

- define what it means for a region of space to be *dense*, making the notion of a cluster boundary parameter-driven rather than geometrically fixed.
- propagate cluster membership through chains of close points, so that arbitrarily shaped clusters emerge from local density alone.
- handle graph structure directly, so that connectivity rather than distance governs which points belong together.

Hands On Experience

CONSIDER the same six canonical points, now with a seventh point $x_7=(4,5)$ placed between the two natural groups.

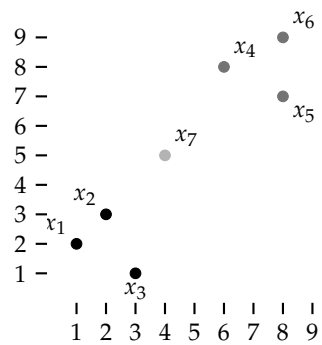


Figure 48: Six canonical points plus $x_7=(4,5)$ sitting between the two natural groups. Which points lie in dense neighbourhoods, and which sits in empty space?

DRAW A CIRCLE of radius $\varepsilon=2.5$ around each point. Which points have at least two neighbours inside their circle? Which point has no neighbours at all? If you were forced to assign x_7 to one of the two groups, which would you choose, and would you trust that assignment?

NOISE SEEMS LIKE THE RIGHT ANSWER for x_7, but could you argue otherwise? What would happen if you enlarged ε to 4.5? And if every isolated point is noise, is there ever a reason to absorb it into a cluster?

WITHOUT FIXED PARAMETERS, there is no single right answer, as a choice of ε and minPts for any answer could be engineered. However, some answers are better than others, and most people agree that x_7 is better left as noise than silently forced into whichever group happens to be nearest.

THE FUNDAMENTAL QUESTION this chapter answers is: how do we formalize the notion of a dense neighbourhood so that cluster membership propagates naturally, noise is identified automatically, and no value of K needs to be chosen in advance?

WHERE WOULD YOU PLACE the boundary between dense and sparse in this data? Could you think of a way to read that boundary off the data itself, rather than guessing it?

THE NEIGHBOURHOOD RADIUS ε is the distance within which a point looks for companions. Small ε finds only very tight clusters and labels most points noise. Large ε absorbs everything into one cluster. The right value lives in between, where the data transitions from dense to sparse. We will see a principled way to find it using the k-distance plot.

The Learning Objectives of this lecture:

- Understand how DBSCAN defines density through ε-neighbourhoods and minimum point counts, and trace the full clustering of a small dataset by hand.
- Apply spectral clustering by constructing the similarity graph, computing the Laplacian, and explaining why the eigenvector embedding separates non-convex clusters.
- Identify when density-based and graph-based methods succeed where K-Means and hierarchical clustering fail, and recognise their own failure modes.

The DBSCAN Algorithm

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) builds clusters from local density. Two parameters govern the algorithm: ε (the neighbourhood radius) and minPts (the minimum number of points required to form a dense region).

DBSCAN. Introduced by Ester, Kriegel, Sander, and Xu in 1996. One of the most cited clustering algorithms, and a standard benchmark for density-based methods.

Every point is classified into one of three roles before any cluster is formed. A point p is a **core point** if its ε-neighbourhood contains at least minPts points:

$$N_\varepsilon(p) = \{q \in X : d(p, q) \leq \varepsilon\}$$

$$p \text{ is core} \iff |N_\varepsilon(p)| \geq \text{minPts}$$

A point q is a **border point** if it is not a core point but falls within the ε-neighbourhood of some core point. A point is **noise** if it is neither core nor border.

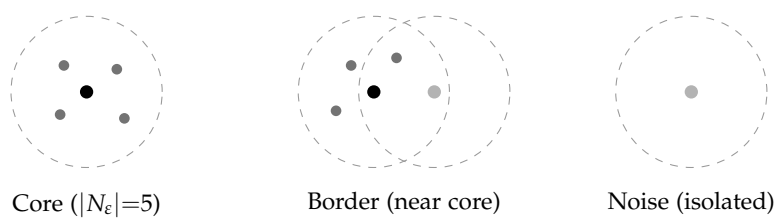


Figure 49: DBSCAN point classification with minPts=4. The black core point has four neighbours within ε (dashed circle). The light grey border point is inside a core's neighbourhood but lacks enough neighbours of its own. The isolated light grey noise point has no core nearby.

Density reachability is not symmetric. A border point q is reachable from core point p (because $q \in N_\varepsilon(p)$), but p is not reachable from q (because q is not core). Cluster membership propagates from cores outward, never in reverse.

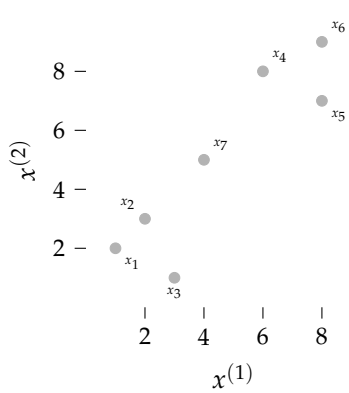
Density connectivity extends reachability to full cluster membership. Point q is **density-reachable** from p if there exists a chain $p{=}p_1, p_2, \ldots, p_n{=}q$ where each $p_{i+1} \in N_\varepsilon(p_i)$ and each p_i is a core point. Two points are **density-connected** if there exists a third point o from which both are density-reachable. A cluster is a maximal set of density-connected points.

Algorithm 3: DBSCAN

Require: Data X, neighbourhood radius ε, density threshold minPts
Ensure: Cluster label for each point (or noise)

```
Mark all points as unvisited; C ← 0
for each unvisited point p ∈ X do
    Mark p as visited; compute N ← N_ε(p)
    if |N| < minPts then
        Label p as noise
    else
        C ← C + 1; assign p to cluster C
        for each q ∈ N \ {p} do
            if q is unvisited then mark visited; compute N′ ← N_ε(q)
                if |N′| ≥ minPts then N ← N ∪ N′
                end if
            end if
            if q has no cluster then assign q to cluster C
            end if
        end for
    end if
end for
```

Trace through our seven points. Dashed circles are ε=2.5 neighbourhoods; dark points belong to C_1, grey to C_2, and an open circle marks noise. Each figure advances one step.



Visit order does not matter. The set of clusters and their memberships is the same regardless of the order in which points are processed. Any unvisited core point triggers a new cluster; border points are absorbed by whichever core reaches them first.

Figure 5b:

$d(x_1, x_2) = \sqrt{1+1} = \sqrt{2} \approx 1.41$ ✓

$d(x_1, x_3) = \sqrt{4+1} = \sqrt{5} \approx 2.24$ ✓

$d(x_1, x_4) = \sqrt{9+1} = \sqrt{10} \approx 3.16$ ×

$d(x_1, x_5) = \sqrt{9+16} = \sqrt{25} = 5.00$ ×

$d(x_7, x_4) = \sqrt{4+9} = \sqrt{13} \approx 3.61$ ×

Start: all $n = 7$ points unvisited.
Parameters $\varepsilon = 2.5$, minPts $= 2$ are fixed.
No cluster has been seeded; every point waits to be processed.
The nearest candidates are both outside ε.

Parameter Selection

Choosing minPts follows a practical rule of thumb: set minPts $\geq d+1$ where d is the number of dimensions. For two-dimensional

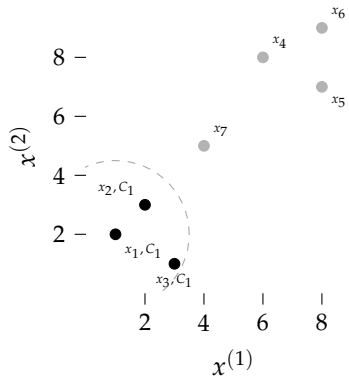


Figure 51:

VISIT x_1: $N_{2.5}(x_1)=\{x_1, x_2, x_3\}$, $|N|=3 \geq 2$, so x_1 is a core point. Seed cluster C_1; assign x_1 to C_1 and add x_2 and x_3 to the expansion queue. Points outside the dashed circle remain unvisited.

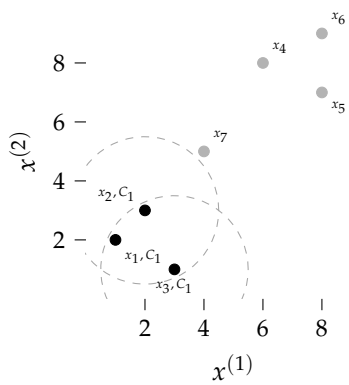


Figure 52:

EXPAND C_1: dequeue x_2: $N_{2.5}(x_2)=\{x_1, x_2, x_3\}$, all already assigned to C_1, no new points added. Dequeue x_3: $N_{2.5}(x_3)=\{x_1, x_2, x_3\}$, same result. Expansion queue empty: $C_1=\{x_1, x_2, x_3\}$ is sealed.

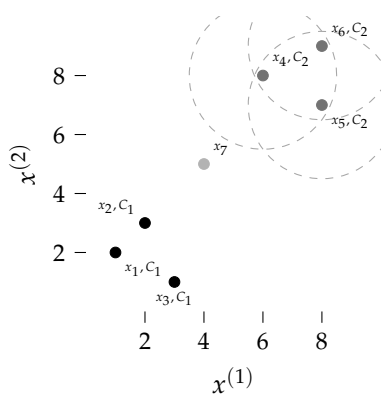


Figure 53:

VISIT x_4: $N_{2.5}(x_4)=\{x_4, x_5, x_6\}$, $|N|=3 \geq 2$, so x_4 is core. Seed C_2; expand from x_5 and x_6: both are core and their neighbourhoods cover only $\{x_4, x_5, x_6\}$. $C_2=\{x_4, x_5, x_6\}$ sealed. Only x_7 remains unvisited.

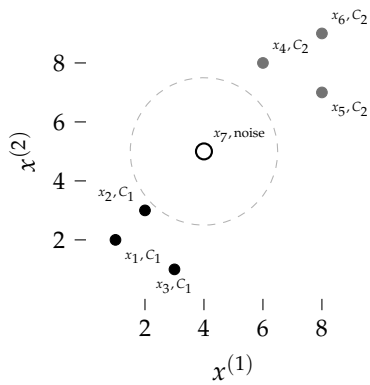


Figure 54:

VISIT x_7: $N_{2.5}(x_7)=\{x_7\}$, $|N|=1<2$, so x_7 is noise. The dashed circle is empty; no neighbour lies within $\varepsilon=2.5$. All points visited. Final result: $C_1=\{x_1, x_2, x_3\}$ (dark), $C_2=\{x_4, x_5, x_6\}$ (grey), noise $=\{x_7\}$ (open circle). No value of K was chosen; noise was never forced into a cluster.

data, minPts=3 or 4 is typical; higher dimensions require larger values to avoid sparse neighbourhoods being treated as dense.

CHOOSING ε uses the k-distance plot: for each point, compute the distance to its k-th nearest neighbour (with $k=\text{minPts}-1$) and sort these distances in ascending order. The curve rises slowly in dense regions and sharply in sparse ones; the elbow of the curve is the recommended ε.

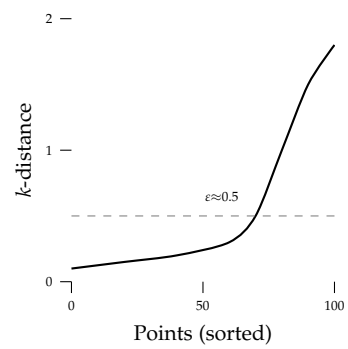
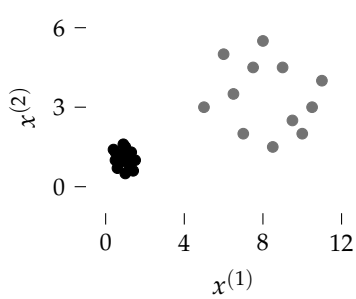


Figure 55: k-distance plot. The elbow at $\approx$0.5 marks the transition from dense to sparse regions.

FAILURE MODES OF DBSCAN

DBSCAN HAS ITS OWN PATHOLOGIES. The two most common arise from the single global ε and from high-dimensional spaces.



VARYING DENSITY

Figure 56: Varying cluster density. An ε tight enough to resolve the left cluster will leave the right cluster disconnected; an ε large enough to connect the right cluster will merge everything on the left into one blob.

HIGH DIMENSIONALITY degrades DBSCAN because all pairwise distances converge as d grows: the ratio of maximum to minimum distance approaches 1, making ε-neighbourhoods either empty or the entire dataset. In practice, DBSCAN is most reliable in two to twenty dimensions.

OPTICS (Ordering Points To Identify the Clustering Structure) addresses varying density by assigning each point a reachability distance: $\text{reach}(p, o) = \max(\text{core-dist}(o),\ d(o, p))$, where core-dist is the distance to the minPts-th nearest neighbour of o. Plotting sorted reachability values produces valleys (dense clusters) and peaks (sparse separations) at every scale simultaneously, removing the need for a single global ε.

When DBSCAN struggles, the root cause is always the same: density is a global threshold applied uniformly across a space that may not be uniform. The remedy is either to adapt the threshold locally (OPTICS) or to abandon distance metrics in favour of graph connectivity (spectral clustering).

HDBSCAN (Hierarchical DBSCAN) takes this further: it sweeps ε from ∞ down to 0, building a hierarchy of all density-based clusterings at once, then extracts the most persistent clusters automatically. No single ε is ever chosen. The result combines the multi-scale view of hierarchical clustering with DBSCAN's ability to handle noise and arbitrary shapes.

Spectral Clustering

Spectral clustering transforms the data into a graph and finds clusters as well-separated subgraphs. It works even when clusters are non-convex, because connectivity rather than geometric proximity determines membership.

The similarity graph is built by computing a weight w_{ij} for every pair of points. The most common choice is the Gaussian kernel:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Points that are close receive weight near 1; distant points receive weight near 0. The bandwidth σ controls how quickly similarity falls off with distance.

The graph Laplacian encodes the graph's connectivity in matrix form. Let W be the $n \times n$ weight matrix and D the diagonal degree matrix with $D_{ii} = \sum_j w_{ij}$. The unnormalized Laplacian is:

$$L = D - W$$

The normalized Laplacian, which corrects for differences in node degree, is:

$$\begin{aligned} L_{\text{sym}} &= D^{-1/2}\, L\, D^{-1/2} \\ &= I - D^{-1/2}\, W\, D^{-1/2} \end{aligned}$$

The algorithm computes the k smallest eigenvectors of L_{sym}, stacks them as columns into a matrix $U \in \mathbb{R}^{n \times k}$, normalises each row of U to unit length, and runs K-Means on the rows.

Why eigenvectors? The number of zero eigenvalues of L equals the number of connected components. The k smallest eigenvectors embed the data so that points in the same component land nearby; K-Means in this embedding finds well-separated clusters even when the original space is non-convex.

Computational Complexity

Mean Shift is a mode-seeking algorithm: each point iteratively moves to the weighted mean of its neighbourhood under a kernel K with bandwidth h. Points converging to the same mode form a cluster. It requires no K and finds non-convex clusters, but the bandwidth h plays the same role as ε and the $O(n^2)$ per-iteration cost limits it to moderate n.

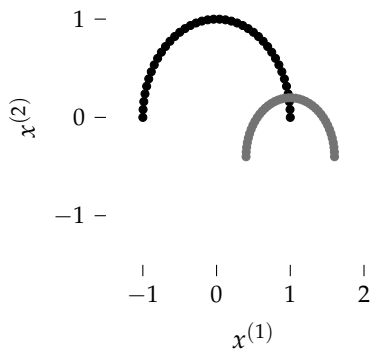


Figure 57: Two interlocking crescents in the original feature space. The clusters are non-convex and inseparable by any straight boundary. No centroid-based method can recover both groups.

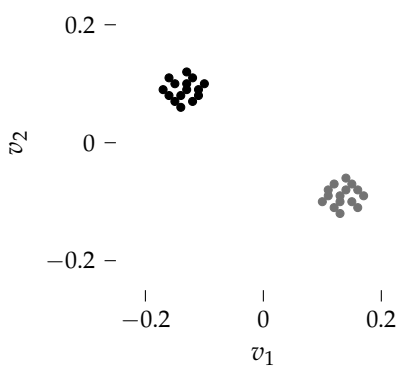


Figure 58: After embedding into the two leading eigenvectors of L_{sym}, the same crescents from Figure 57 are linearly separable and K-Means succeeds.

THE SIMILARITY MATRIX in spectral clustering is the dominant bottleneck. Storing all $\frac{n(n-1)}{2}$ pairwise weights requires $O(n^2)$ memory, and the eigendecomposition of the $n \times n$ Laplacian costs $O(n^3)$. Practical implementations use sparse graphs (connecting each point to only its k nearest neighbours) to reduce both costs substantially.

Algorithm	Time	Space	Notes
K-Means (baseline)	$O(nKTd)$	$O(n+K)$	K and T chosen in advance; no noise concept; fails on non-convex clusters
DBSCAN (naive)	$O(n^2)$	$O(n^2)$	$O(n \log n)$ with k-d tree or ball tree in low dimensions
OPTICS	$O(n^2)$	$O(n)$	No fixed ε; produces full reachability ordering
Mean Shift	$O(n^2T)$	$O(n)$	T iterations; exact cluster count unknown until convergence
Spectral clustering	$O(n^3)$	$O(n^2)$	Eigendecomposition dominates; sparse graph approximations reduce cost

Table 5: Computational complexity of density and graph-based clustering methods, with K-Means listed as a baseline. DBSCAN with a spatial index is practical for large n in low dimensions. Spectral clustering is prohibitive for large n without sparse graph approximations.

Examples & Exercises

COMPUTING BY HAND reveals what the algorithm actually does at each step; run it slowly and feel where density propagates and where it stops.

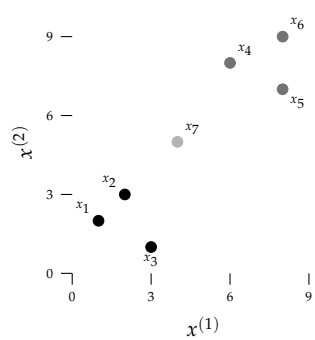


Figure 59: Seven points for the DBSCAN trace. $x_7=(4,5)$ sits between the two natural groups.

GIVEN SEVEN POINTS $x_1=(1,2)$, $x_2=(2,3)$, $x_3=(3,1)$, $x_4=(6,8)$, $x_5=(8,7)$, $x_6=(8,9)$, $x_7=(4,5)$, run DBSCAN with $\varepsilon=2.5$ and minPts=2. Your tasks: compute the neighbourhood size $|N_{2.5}(x_i)|$ for each point, classify each point as core, border, or noise, and identify the clusters.

NEIGHBOURHOOD SIZES. For each point x_i count all points within distance 2.5 (including x_i itself). Here is the full calculation for x_1:

$$
\begin{aligned}
d(x_1, x_2) &= \sqrt{(2-1)^2 + (3-2)^2} \\
&= \sqrt{1+1} \\
&= \sqrt{2} \\
&\approx 1.41 \quad (\leq 2.5, \text{ counted}) \\
d(x_1, x_3) &= \sqrt{(3-1)^2 + (1-2)^2} \\
&= \sqrt{4+1} \\
&= \sqrt{5} \\
&\approx 2.24 \quad (\leq 2.5, \text{ counted}) \\
d(x_1, x_7) &= \sqrt{(4-1)^2 + (5-2)^2} \\
&= \sqrt{9+9} \\
&= \sqrt{18} \\
&\approx 4.24 \quad (> 2.5, \text{ not counted})
\end{aligned}
$$

So $N_{2.5}(x_1) = \{x_1, x_2, x_3\}$ and $|N_{2.5}(x_1)| = 3 \geq 2$: x_1 is a **core point**. Compute the remaining neighbourhood sizes yourself, then verify against the table below.

Point	**Coords**	$\lvert N_{2.5}\rvert$	**Type**	**Neighbours in** $N_{2.5}$	**Cluster**
x_1	$(1,2)$	3	Core	$\{x_1, x_2, x_3\}$	C_1
x_2	$(2,3)$	3	Core	$\{x_1, x_2, x_3\}$	C_1
x_3	$(3,1)$	3	Core	$\{x_1, x_2, x_3\}$	C_1
x_4	$(6,8)$	3	Core	$\{x_4, x_5, x_6\}$	C_2
x_5	$(8,7)$	3	Core	$\{x_4, x_5, x_6\}$	C_2
x_6	$(8,9)$	3	Core	$\{x_4, x_5, x_6\}$	C_2
x_7	$(4,5)$	1	Noise	$\{x_7\}$ only	–

Table 6: DBSCAN trace for ε=2.5, minPts=2. All six canonical points are core points; x_7 has no neighbour within radius 2.5 and is labelled noise. The two natural clusters are recovered without specifying K and without assigning the noise point to either group.

VERIFY x_7 IS NOISE. Compute $d(x_7, x_3)$ to confirm that x_7 lies outside the ε-neighbourhood of its nearest candidate:

$$
\begin{aligned}
d(x_7, x_3) &= \sqrt{(4-3)^2 + (5-1)^2} \\
&= \sqrt{1+16} \\
&= \sqrt{17} \\
&\approx 4.12 \quad (> 2.5)
\end{aligned}
$$

Compute $d(x_7, x_4)$ yourself. Why is x_7 correctly identified as noise rather than being assigned to whichever cluster is nearer?

PARAMETER SENSITIVITY. Re-run the trace with ε=4.5 and minPts=2. Does x_7 change classification? At what minimum value of ε does x_7 first enter the neighbourhood of some core point?

COMPARISON WITH K-MEANS. Run K-Means with K=2 on the same seven points. Which cluster does K-Means assign x_7 to? Is that assignment more or less informative than DBSCAN's noise label, and why?

K-MEANS BASELINE RESULT. After convergence with K=2, the centroids settle at $\mu_1 \approx (2.0,\ 2.0)$ and $\mu_2 \approx (7.3,\ 8.0)$. Point x_7=$(4, 5)$ is closer to μ_1:

$$\begin{aligned} d(x_7,\ \mu_1) &= \sqrt{(4-2)^2 + (5-2)^2} \\ &= \sqrt{4+9} \\ &= \sqrt{13} \approx 3.61 \end{aligned}$$

$$\begin{aligned} d(x_7,\ \mu_2) &= \sqrt{(4-7.3)^2 + (5-8.0)^2} \\ &= \sqrt{10.89+9} \\ &= \sqrt{19.89} \approx 4.46 \end{aligned}$$

K-Means assigns x_7 to C_1, as shown below. The assignment is not wrong in a strict sense; it is simply forced. K-Means has no noise concept and cannot abstain.

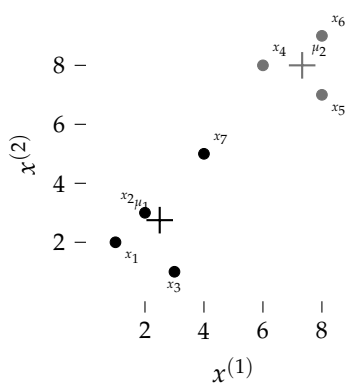


Figure 60: K-Means baseline with K=2. Crosses mark centroids μ_1 and μ_2. Because K-Means must assign every point, x_7 is forced into C_1 (dark). Compare with Figure 54: DBSCAN labels x_7 noise and makes no forced assignment.

A GEOSPATIAL DATASET TO EXPLORE. Apply DBSCAN to a set of GPS coordinates (e.g., city check-in records or taxi pickup points).

CODE will be provided as a Python notebook.

WHAT TO OBSERVE. Try several (ε, minPts) combinations and note how the number of clusters and noise points change. Plot the k-distance curve before choosing ε, and check whether the elbow matches the value that gives the most interpretable clustering.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- Why can DBSCAN find clusters of arbitrary shape while K-Means cannot?
- What happens to a border point if minPts is increased by one?
- Why is density reachability not symmetric, and what does that asymmetry imply for cluster assignment?
- DBSCAN with a very large ε and minPts=1 behaves like which simpler algorithm?
- OPTICS removes the need for a fixed ε; what does it require from the user instead?
- In what sense does spectral clustering use K-Means internally, and why is that not a contradiction?
- Why does the number of zero eigenvalues of the Laplacian reveal the number of connected components?
- When would you prefer DBSCAN over hierarchical clustering, even if both recover the correct partition?

RECAP of Key Concepts:

- **DBSCAN** classifies every point as core, border, or noise using ε and minPts, and builds clusters by propagating density reachability from core points outward without specifying K.
- **Noise handling** is native: any point with fewer than minPts neighbours within ε is labelled noise and excluded from all clusters.
- **OPTICS** removes the single global ε by producing a reachability plot that exposes cluster structure at every density scale simultaneously.
- **Spectral clustering** embeds data into the eigenvectors of the graph Laplacian, making non-convex clusters linearly separable before applying K-Means.
- **Parameter sensitivity** is the shared weakness: ε and σ must be chosen carefully, and both methods degrade in high dimensions where all distances become indistinguishable.

THE ROOT OF DENSITY FAILURE. Every density-based method sets a single threshold on what counts as close enough. When true cluster density varies across the dataset, no fixed value works everywhere.

Density-based and graph-based methods both escape the spherical cluster assumption, but they still operate in the original feature space. With ten features the ε-ball captures something meaningful; with a thousand features it either captures everything or nothing. The same data that breaks DBSCAN in high dimensions also makes the similarity graph in spectral clustering nearly uniform, erasing the connectivity structure that the eigenvectors are supposed to encode.

The underlying problem is that the data occupies a high-dimensional space but the meaningful variation lives in far fewer dimensions. Before we can cluster reliably at scale, we need to find those dimensions.

Teaser If cluster structure lives in a low-dimensional subspace, can we find that subspace without knowing the clusters first? And can we do it in a way that preserves the global geometry, the local geometry, or both?

Index

agglomerative clustering, 44
average linkage, 48

BIRCH, 50
border point, 63

Centroid Clustering, 26
centroid clustering, 27
  limitations, 41
chaining
  single linkage, 49
cluster, 29
clustering, 29
complete linkage, 48
computational complexity
  density clustering, 67
  hierarchical clustering, 50
core point, 63
Cosine similarity, 17
covariance matrix, 18
curse of dimensionality, 66

data point, 29
data preprocessing, 19
DBSCAN, 59, 63
  parameter selection, 64
dendrogram, 41, 42
  leaf ordering, 54
  vocabulary, 51
density connectivity, 63
density-based clustering, 59
distance, 13
distance metric, 16
Distance Metrics, 13
divisive clustering, 43

elbow method, 33
Euclidean distance, 16
examples, 21, 35, 53, 69
exercises, 21, 35, 53, 69

failure modes, 34
  DBSCAN, 66
  hierarchical clustering, 49
feature space, 16
Foundations, 13

graph Laplacian, 67

HDBSCAN, 67
hierarchical clustering, 41
  new data, 52

inertia, 29
irrecoverability
  hierarchical clustering, 49

k-distance plot, 66
K-Means, 26, 27, 29
K-Means++, 31

L1 distance, 17
Lance-Williams formula, 56
license, 2
linkage criterion, 48

Mahalanobis distance, 18
Manhattan distance, 17
mean shift, 67
Min-Max scaling, 20

noise point, 63
normalization, 19

OPTICS, 66

recap, 25, 38, 57, 73
reflection, 25, 38, 57, 73

scalability
  hierarchical clustering, 50
shape bias
  Ward's linkage, 49
silhouette score, 33
similarity, 13
similarity graph, 67
single linkage, 48
  complexity, 50
SLINK, 55
spectral clustering, 59, 67
standardization, 19

unsupervised learning, 13

Ward's method, 49
  variance increase, 51

Z-score normalization, 19